

CHAPTER

6

BUILDING WEB SERVICES

Learning Objectives

6.1 INTRODUCTION

6.2 INTRODUCTION TO WEB SERVICES

- 6.2.1 Applications of Web Service
- 6.2.2 Key Web Service Technologies
- 6.2.3 Web services advantages
- 6.2.4 Web Services Limitations

6.3 ELEMENTS OF WEB SERVICES

6.4 WEB SERVICE INFRASTRUCTURE

6.5 SIMPLE OBJECT ACCESS PROTOCOL (SOAP)

6.6 BUILDING A WEB SERVICE

6.7 DEPLOYING WEB SERVICE

- 6.7.1 Web service deployment tools

6.8 PUBLISHING WEB SERVICES

6.9 FINDING WEB SERVICES

6.10 CONSUMING WEB SERVICES

REVIEW QUESTIONS

6.1 INTRODUCTION

Throughout this book, we have created dynamic link libraries (DLLs) to facilitate software reusability and modularity—the cornerstones of good object-oriented programming. However, the use of DLLs is limited by the fact that DLLs must reside on the same machine as the programs that use them. This chapter introduces the use of Web services (sometimes called XML Web services) to promote software reusability in distributed systems. Distributed systems technologies allow applications to execute across multiple computers on a network. A Web service is an application that enables distributed computing by allowing one machine to call methods on other machines via common data formats and protocols, such as XML and HTTP. In .NET, these method calls are implemented using the Simple Object Access Protocol (SOAP), an

XML-based protocol describing how to mark up requests and responses so that they can be transferred via protocols such as HTTP. Using SOAP, applications represent and transmit data in a standardized format—XML. The underlying implementation of the Web service is irrelevant to clients using the Web service.

Microsoft is encouraging software vendors and e-businesses to deploy Web services. As more and more people worldwide connect to the Internet via networks, applications that call methods across a network becomes more practical. Earlier in chapter 3, we discussed the concepts of object-oriented programming. Web services represents the next step in object-oriented programming: Instead of developing software from a small number of class libraries provided at one location, programmers can access countless libraries in multiple locations.

6.2 INTRODUCTION TO WEB SERVICES

"A Web Service is a software program that uses XML to exchange information with other software via common internet protocols. In a simple sense, Web Services are a way for interacting with objects over the Internet."

A web service is

- Language Independent.
- Protocol Independent.
- Platform Independent.
- It assumes stateless service architecture.
- Scalable (e.g. multiplying two numbers together to an entire customer-relationship management system).
- Programmable (encapsulates a task).
- Based on XML (open, text-based standard).
- Self-describing (metadata for access and use).
- Discoverable (search and locate in registries)- ability of applications and developers to search for and locate desired Web services through registries. This is based on UDDI.

A Web service is an application stored on one machine that can be accessed on another machine over a network. Due to the nature of this relationship, the machine on which the Web service resides commonly is referred to as a remote machine. The application that accesses the Web service sends a method call to the remote machine, which processes the call and sends a response to the application. This kind of distributed computing benefits various systems, including those without access to certain data and those lacking the processing power necessary to perform specific computations.

A Web service is, in its simplest form, a class. In previous chapters, when we wanted to include a class in a project, we would either define the class in our project or add a reference to a compiled DLL. This compiled DLL is placed in the bin directory of an application by default. As a result, all pieces of our application reside on one machine. When using Web services, the

class (and its compiled DLL) we wish to include in our project are stored on a remote machine—a compiled version of this class is not placed in the current application.

Methods in a Web service are remotely invoked using a Remote Procedure Call (RPC). These methods, which are marked with the WebMethod attribute, often are referred to as Web-service methods. Declaring a method with this attribute makes the method accessible to other classes via an RPC. The declaration of a Web-service method with attribute WebMethod is known as exposing the method, or enabling it to be called remotely.

Web Services basically uses Hypertext Transfer Protocol (HTTP) and SOAP to make business data available on the Web. It exposes the business objects (COM objects, Java Beans, etc.) to SOAP calls over HTTP and executes remote function calls. The Web Service consumers are able to invoke method calls on remote objects by using SOAP and HTTP over the Web.

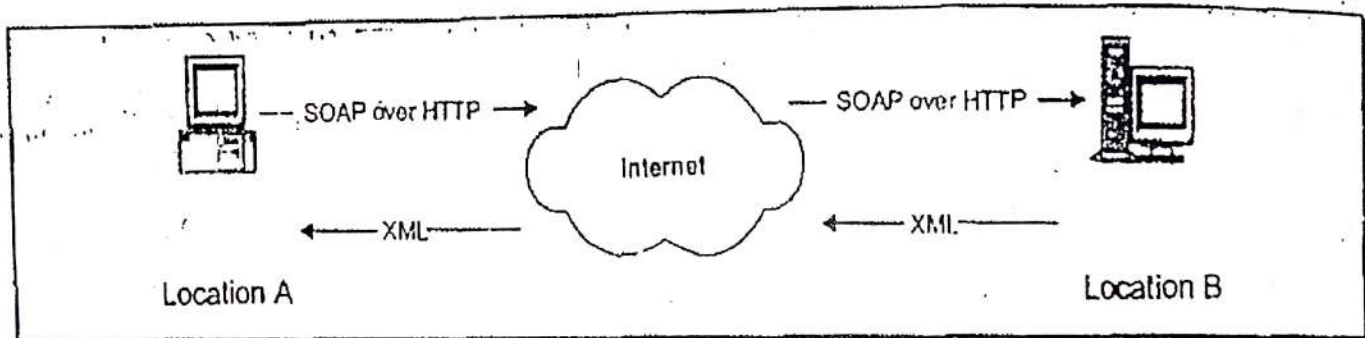


Figure 6.1 SOAP calls are remote function calls that invoke method executions on Web Service components at Location B. The output is rendered as XML and passed back to the user at Location A.

How is the user at Location A aware of the semantics of the Web Service at Location B? This question is answered by conforming to a common standard. Service Description Language (SDL), SOAP Contract Language (SCL) and Network Accessible Specification Language (NASSL) are some XML-like languages built for this purpose. However, IBM and Microsoft recently agreed on the Web Service Description Language (WSDL) as the Web Service standard.

Most requests to and responses from Web services are transmitted via SOAP (discussed in section 6.4). This means that any client capable of generating and processing SOAP messages can use a Web service, regardless of the language in which the Web service is written. Web services have important implications for business-to-business (B2B) transactions, (i.e., transactions that occur between two or more businesses).

Now, instead of using proprietary applications, businesses can conduct transactions via Web services—a much simpler and more efficient means of conducting business. Because Web services and SOAP are platform-independent, companies can collaborate and use Web services without worrying about the compatibility of various technologies or programming languages. In this way, Web services are an inexpensive, readily-available solution to facilitate B2B transactions. A Web service created in Visual Studio .NET has two parts: An ASMX file and a codebehind file. The ASMX file by default can be viewed in any Web browser and contains valuable information about the Web service, such as descriptions of Web-service methods and

ways to test these methods. The code-behind file provides the implementation for the methods that the Web service encompasses.

In short, a Web service is a programmable URL. Stated another way, a Web service is an application component that is remotely callable using standard Internet protocols such as HTTP and XML. Thus, any system that supports these basic, standard protocols is capable of supporting Web services.

6.2.1 Applications of Web Service

There are many potentially valuable applications of Web services. Some of these include:

- Weather Reporting: You can use Weather Reporting web service to display weather information in your personal website.
- Stock Quote: You can display latest update of Share market with Stock Quote on your web site.
e.g. example 7 Gold & Silver etc
- News Headline: You can display latest news update by using News Headline Web Service in your website.
- Services that are either too difficult or too expensive to implement yourself. For example, credit card validation, financial account management, stock quotes, and so on.
- Services that provide commonly needed functionality for other services. For example, user authentication, usage billing, usage auditing, and so on.
- Services that aggregate distributed, discrete services into an orchestrated whole. A good example of this type of service would be travel booking.
- Services that integrate your business systems with your partners (or other business systems within your own organization).

2.2 Key Web Service Technologies

- **XML**- Describes only data. So, any application that understands XML-regardless of the application's programming language or platform-has the ability to format XML in a variety of ways (well-formed or valid).
 - **SOAP**- Provides a communication mechanism between services and applications.
 - **WSDL**- Offers a uniform method of describing web services to other programs.
 - **UDDI**- Enables the creation of searchable Web services registries.
- When these technologies are deployed together, they allow developers to package applications services and publish those services on a network. The more information regarding these technologies are in topic 6.3.

2.3 Web services advantages

There are following advantages of web services:

- Use open, text-based standards, which enable components written in various languages and for different platforms to communicate.
- Promote a modular approach to programming, so multiple organizations can communicate with the same Web service.

- Comparatively easy and inexpensive to implement, because they employ an existing infrastructure and because most applications can be repackaged as Web services.
- Significantly reduce the costs of enterprise application (EAI) integration and B2B communications.
- Implemented incrementally, rather than all at once which lessens the cost and reduces the organizational disruption from an abrupt switch in technologies.
- The Web Services Interoperability Organization (WS-I) consisting of over 100 vendors promotes interoperability.

6.2.4 Web Services Limitations

There are following limitations of web services:

- SOAP, WSDL, UDDI- require further development.
- Interoperability.
- Royalty fees.
- Too slow for use in high-performance situations.
- Increase traffic on networks.
- The lack of security standards for Web services.
- The standard procedure for describing the quality (i.e. levels of performance, reliability, security etc.) of particular Web services & management of Web services.
- The standards that drive Web services are still in draft form (always will be in refinement).
- Some vendors want to retain their intellectual property rights to certain Web services standards.

6.3 ELEMENTS OF WEB SERVICES

Now that you have an idea of what Web services are and how they can be used, let's examine the key technologies that you will encounter when working with Web services. The key to the broad-reach capabilities of Web services on the Internet is a foundation built on Internet standards that does not rely on any platform-specific technology. This foundation supplies standards-based services that provide the following capabilities to Web services:

- A standard, portable method for describing data
- A standard, extensible message format for communicating requests and responses
- A standard, extensible method for describing the capabilities of Web services
- A method to discover what Web services are available at any particular site
- A method to discover what sites provide Web services

The following points will help you to understand why these issues are important and also introduce you to the technologies that provide these capabilities to the Web service platform.

6.3.1 Representing Data

Web services enable consumers to programmatically request and obtain structured data. But, how is this data encoded so that it can be exchanged between service and consumer? How do you ensure a consistent and accurate interpretation of the data when the service and consumer may reside on different platforms, operating systems, object models, and/or programming languages?

To promote the ability of Web services to communicate their data unambiguously, efficiently, and effectively, a common, portable, and standard method for describing data must be used.

The simple (and logical) answer is XML. As you will soon see, XML is used extensively in every aspect of Web services. The following is an introduction to the specific XML technologies you will encounter when building or consuming Web services.

XML: XML provides a standards-based method for describing data (also known as metadata). XML has the ability to describe data using a simple grammar that is highly interoperable among the many heterogeneous systems that are connected to the Internet. Using the basic elements of the XML language, you can describe simple as well as complex data types and relationships. XML has several key strengths that have helped it to become the de facto method for describing data (especially over HTML and other binary formats):

- XML is a text-based language, which makes it easily readable and more portable than binary data formats.
- XML gives you the ability to define your own tags to describe data and its relationships to other data (hence, the word extensible in its name).
- XML strictly enforces its language syntax, unlike HTML.
- Parsers are widely available to accurately parse and validate XML structures that you define, which means you don't have to do it yourself!

Of course, XML is not without its weaknesses. Among these is the fact that XML can be more "wordy" than an equivalent binary format. In addition, data interoperability can be difficult to achieve due to differences in naming conventions from one application to another.

6.3.2 Exchanging Messages

Web services communicate in the form of messages. A request message delivers information about an operation to be executed and any data required to carry out that operation. Request messages flow from clients to Web services. A response message delivers information about the results of the operation execution. Response messages flow from Web services to clients.

Communication via messages is an extremely effective method for insulating Web service consumers from the implementation details of the service. Of course, it is necessary to define, in a standard way, the rules for how these messages should be formatted and what they can contain.

-Message exchange with SOAP: The Simple Object Access Protocol (SOAP) is an industry-standard message format that enables message-based communication for Web services. SOAP implements a message format based on XML to exchange function requests and responses.

BUILDING WEB SERVICES

Using XML as the basis for SOAP messages makes it possible for these messages to be understandable and transportable by any system that implements basic Internet communications services.

SOAP simply defines a message format. It does not impose a specific transport protocol for exchanging these messages. Thus, it is possible to transport SOAP messages over any one of many different and widely available transport protocols such as HTTP, SMTP, FTP, and so on.

6.3.3 Web Service Capabilities

A Web service description is an XML document that defines the capabilities that a Web service offers. This document provides essential information in a structured form that enables a consumer to understand how to interact with a Web service. The Web Service Description Language (WSDL) defines a standard, extensible XML grammar that is used to define these Web service descriptions in the form of an XML document.

The WSDL document defines the message formats and message exchange patterns that a Web service can process. In addition to these definitions, the WSDL document contains the address of each Web service entry point, formatted according to the protocol used to access the service (for example, a URL for HTTP or an e-mail address for SMTP).

6.3.4 Publishing and Finding Web Services

Given a standard means to describe the capabilities of a Web service via the WSDL document, we must now consider how a potential consumer of a Web service will locate a WSDL document on a target Web server. To consume a service, a client must be able to determine how to interact with that Web service, which means that it must follow the message formats and message exchange patterns described for the Web service in the WSDL document.

Of course, if you are both the author and consumer of the Web service, you probably will not need help in locating the WSDL document. However, if you will be consuming Web services from other authors, you may not know where a particular service is located on a target Web server. Web service authors use a discovery (DISCO) document to publish their Web service. The DISCO document is an XML document that contains pointers to such things as the WSDL file for the Web service.

Web service consumers employ a discovery process to learn that a Web service exists and where to find the Web service's WSDL document. Web service consumers enact this discovery process on a target Web server by providing a URL to a discovery tool. The discovery tool attempts to locate DISCO documents on the target server and informs the consumer of the location of any available WSDL documents.

6.3.5 Finding Web Service Providers

As more and more Web services are created and deployed by numerous organizations on the Internet, it will become increasingly more difficult for consumers to find these services. This is analogous to the difficulty Web users might experience in finding a specific page of information amongst the billions of pages of information currently published on the Web were it not for Web search engines.

Similar to the search engine approach that is used to query and locate Web pages, the Universal Description, Discovery, and Integration (UDDI) specification defines a logically centralized but physically distributed, XML-based registry database and API for companies to find each other and the Web services that they offer. The UDDI registry API offers support for querying as well as updating the registry database. UDDI defines, classifies and stores three basic types of information:

White Pages: Describes address, contact, and other standard business demographic information

Yellow Pages: Describes industrial categorizations for businesses based on standard categories

Green Pages: Describes the technical specification for Web services.

6.4 WEB SERVICE INFRASTRUCTURE

Four primary infrastructure pieces are specifically needed to support Web services. Collectively, these technologies provide services that are essential to the creation and consumption of Web services.

- **Web Service Directories**, which provide a means to locate providers of Web services
- **Web Service Discovery**, which provides the capability to locate Web services
- **Web Service Description**, which enables Web service capabilities to be described
- **Web Service Wire Formats**, which allow Web services to exchange data and messages

Web Service directories *store data*

*Short of my 1001
new 1001 for 2 days*

Web service directories provide a centralized, Internet-accessible location that consumers can use to find Web services that have been offered for use by other companies or organizations. You can think of Web service directories as a type of Web portal or "Yellow Pages" specifically suited for listing and locating Web services. Using Web service directories, you can search for Web services using a variety of structured criteria, such as business type, industry, type of goods produced, services offered, and so on. For example, if you were looking for a credit card validation Web service, you could search the directory using personal credit companies as the criteria. Currently, the Universal Description, Discovery, and Integration (UDDI) specification is the de facto standard for cataloging and finding Web services.

Web Service discovery

Particular service (WSDL) format

Web service discovery is the process of locating one or more related documents that describe specific Web service. Web services are described in terms of the request messages they are capable of processing and the response messages (if any) that they return. These capabilities are described in a standard way using the Web Service Description Language (WSDL), which is an XML grammar specifically designed for this purpose.

Before you can submit requests to a Web service, you must know how to format requests for particular service in the form of a message that encodes the operation requested (such as converting a temperature from one unit to another) as well as any data required to carry out the

operation (such as the input temperature, the source units, and the target units). In addition, you must know whether or not to expect a response message from the Web service and what format this response will take (such as the converted temperature value).

The Web service discovery process permits a consumer to search for and locate the WSDL document for a Web service. As implied, a consumer must have possession of the WSDL document before any requests can be properly formatted and delivered to the Web service. The DISCO specification defines an XML-based grammar and algorithm for discovering the existence of Web services and locating their WSDL document. Using DISCO, you can define search trees that are processed according to the DISCO algorithm to locate Web service descriptions. Of course, if you already know the location of the WSDL document for a specific Web service, this discovery process is not needed.

Web Service description

A Web service description is an XML document that defines the capabilities of a Web service. Using the WSDL XML grammar, you can clearly and unambiguously define the Web-addressable entry points in terms of the request messages that a Web service will accept as well as the response messages a Web service can return. Also included in this description are the supported protocol bindings and a description of the data types processed by the Web service.

.NET Framework supports self-describing assemblies. This is accomplished by storing metadata with the assembly that describes the interfaces, data types, and other information about the classes in the assembly. Using the self-describing nature of .NET assemblies, the .NET Framework can generate WSDL documents to describe Web service capabilities from the .NET assemblies that contain ASP.NET Web service code. Much of this support is built into the System.Web.Services.Description namespace in the .NET Framework.

From the Web service perspective, ASP.NET supports the dynamic generation of WSDL documents from the Web Service assembly when it is requested. This eliminates issues related to keeping a separate WSDL document in sync with the Web Service assembly that implements the service.

Web Service Wire Formats/Transport

The final piece of the ASP.NET Web Services infrastructure consists of the Web service wire formats. Wire formats define the method by which Web service request and response messages are encoded and transported between the Web service and any consumer. To maximize the reach of Web services on the Internet, standard Internet protocols are used.

ASP.NET Web Services support three wire formats: HTTP-GET, HTTP-POST, and HTTP-SOAP. Traditional Web applications have used HTTP-GET and HTTP-POST to deliver Web forms-based data to the Web server for processing. These same protocols are used to deliver Web service operation requests along with any necessary arguments to the Web service for processing. The HTTP-SOAP wire format is a new format that has been developed exclusively for enabling Web services to communicate very rich data types. Each of these wire formats finds its implementation in the System.Web.Services.Protocols namespace of the .NET Framework Class Library. Let's take a look at how these wire formats are implemented for ASP.NET Web Services.

6.10 (i) HTTP-GET

The HTTP-GET protocol encodes Web service operation requests and arguments in the URL to the Web service. The operation is coded as part of the URL string and any arguments are coded as query string parameters appended to the base URL. For example:

http://localhost/ctemp/ctemp.asmx/ctemp?Temperature=32&FromUnits=F&ToUnits=C

This URL specifies the Web-addressable entry point for the CTemp Web service (Ctemp.asmx), including the method to be called (also named ctemp). The arguments to the ctemp method are passed as query string arguments to the method request. Similar to the way in which WSDL documents are generated and returned to requests for such information via a URL to the Web service entry point file (the ASMX file), the HTTPGET method of calling Web service methods is handled by the WebServiceHandlerFactory class. This class takes the URL and query string parameters as input and translates this into a method call on the appropriate Web service class implementation.

(ii) HTTP-POST

The HTTP-POST protocol encodes Web service operation requests and arguments within the payload area of the HTTP-POST request as name/value pairs. This technique of invoking a Web service method is identical in operation to the HTTPGET method (from an ASP.NET perspective), except in the way in which the Web service call arguments are passed to the server.

Once again, the .NET Framework's WebServiceHandlerFactory class is responsible for extracting the method name and arguments from the request and calling the appropriate Web service method found in the Web service class implementation.

.NET Proxy classes use the HttpPostClientProtocol class in the System.Web.Services.Protocols namespace to invoke Web services that support the HTTP-POST protocol. HTTP-SOAP

(iii) HTTP-SOAP

It is the default ASP.NET Web Service wire format. It is based on the SOAP specification (currently submitted to the W3C as a note) and supports the widest range of simple and complex data types (including document-oriented operations). Web service request and response messages are encoded into SOAP messages that are included in the payload area of an HTTP-POST message. SOAP messages are encoded in XML using the SOAP vocabulary defined in the specification. Because SOAP is really XML, it is possible to describe nearly any type of data. This makes SOAP an excellent choice for passing rich data types between Web services and their consumers. For example, it is possible to pass very complex types such as entire XML documents, such as an invoice or purchase order. .NET Proxy classes use the HttpSimpleClientServiceProtocol class in the System.Web.Services.Protocols namespace to invoke Web services that support the HTTP-SOAP protocol.

6.5 SIMPLE OBJECT ACCESS PROTOCOL (SOAP)

SOAP is a lightweight, XML-based protocol for exchanging information in a decentralized, distributed environment, such as that offered by the Internet. In other words, SOAP enables two

processes (possibly on different machines) to communicate with each other regardless of the hardware and software platforms on which they are running.

The fundamental building block of SOAP is XML. SOAP defines a specialized, yet flexible XML grammar that standardizes the format and structure of messages. Messages are, in turn, the fundamental method for exchanging information between Web services and Web service consumers. Using XML to encode SOAP messages provides several benefits, such as these:

- XML is human readable, making it easier to understand and debug.
- XML parsers and related technologies are widely available.
- XML is an open standard.
- XML includes many related technologies that can be leveraged in SOAP.

Thus, XML is a natural choice for encoding SOAP messages and contributes to the simplicity of the specification. Typically, a Web service consumer will send a message to a Web service, requesting a specific operation to be performed. The Web service processes this request and typically (but not necessarily) returns the results in a response message. This request/response model is conceptually akin to the Remote Procedure Call (RPC) model.

To transport SOAP messages, we need a transport protocol. The obvious choice for a transport protocol is HTTP, because it is in use on so many systems today, making it widely available. In addition, HTTP is typically allowed through most firewalls today, making it easy to get up and running without requiring administrators to open more ports through their corporate firewalls.

Although HTTP is an obvious choice for a transport protocol (and the one that most major vendors are implementing), the SOAP specification does not require a specific transport protocol. It is quite possible to transport SOAP messages over other transport mechanisms such as SMTP and FTP. However, the default transport protocol for ASP.NET Web Services based on SOAP is HTTP.

So, in a nutshell, SOAP provides the following capabilities:

- Enables interoperability between systems using standard, widely available protocols such as XML and HTTP.
- Allows systems to communicate with each other through firewalls, without having to open additional, potentially unsafe ports.
- SOAP fully describes each data element in the message, making it easier to understand and troubleshoot problems that may occur.

Arguably, as important as what SOAP addresses to enable interoperability is what it does not attempt to address. Specifically, SOAP does not do the following:

- Attempt to define how objects are created or destroyed.
- Impose any specific security mechanism or implementation.
- Define an authentication scheme

6.4.1 SOAP Message Elements

A SOAP message is composed of three primary elements, each of which performs a special purpose.

Envelope: Serves as a container for the remaining SOAP message elements.

Header: Contains optional data a consumer may or may not be required to understand to process the message properly. This is the primary extensibility mechanism of SOAP.

Body: Contains the actual encoding of a method call and any input arguments or an encoded response that contains the results of the method call.

The SOAP envelope

The SOAP envelope element is a required part of a SOAP message. It serves as a container for all remaining SOAP message elements. Typically, this includes the SOAP header and body elements. In addition to serving as a container for the header and body elements, the envelope defines the namespaces used by these elements. Figure 6.1 graphically depicts the structure of a complete SOAP message.

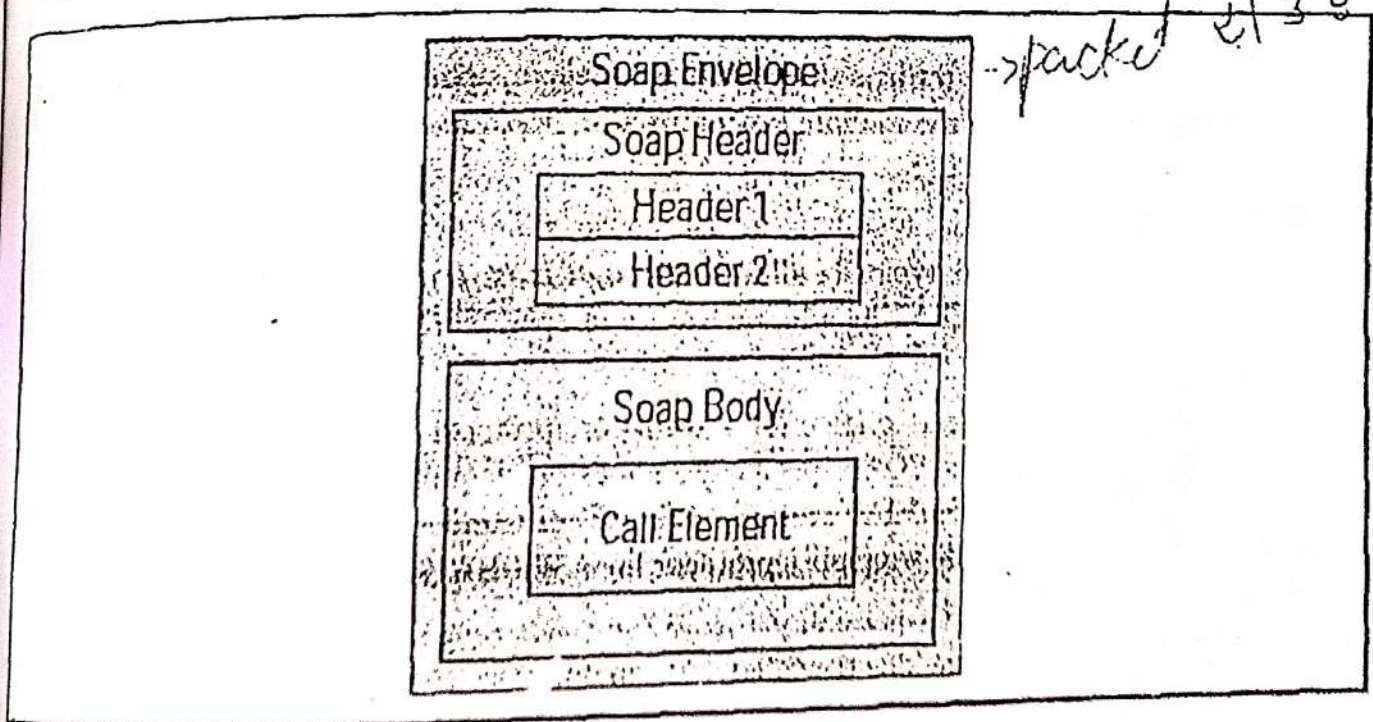


Figure 6.1: Structure of a SOAP Message

To deliver a SOAP message to an endpoint, addressing information specific to the transport protocol binding is used as a means to ensure that the message is delivered to the correct endpoint, much like the name and address on a real envelope that is used by a postal authority to deliver the mail. In the case of HTTP, a custom HTTP header named SOAPAction is used to direct the message to the proper endpoint. One of the major reasons that message addressing is implemented in this manner is that systems administrators can configure firewall software to look for and filter traffic based on this header information, without requiring parsing of the XML.

The SOAP header

The SOAP header element is an optional part of a SOAP message. It defines additional information that can be related to the method request in the body element or, more likely, information that is independent of the method request that is required or otherwise useful to your application. SOAP does not define the specific contents or semantics for a SOAP header.

SOAP headers are quite similar in concept to the META tags found in HTML documents. They

define metadata that can be used to provide context to, or otherwise direct the processing of, the message.

The SOAP body

The SOAP body element is a required part of a SOAP message that contains the data specific to a particular method call, such as the method name and any input/output arguments or the return values produced by the method. The contents of the SOAP body depend on whether the message is a request or a response. A request message contains method call information; whereas a response message contains method call result data.

6.6 BUILDING A WEB SERVICE

In ASP.NET, the complexities of SOAP messaging and HTTP transport are abstracted from developers, so creating XML-based Web services is actually quite simple. This abstraction is enabled through attributes that tell the common language runtime (CLR) to treat a specified class and its methods as Web services. Based on these attributes, the CLR provides all of the plumbing necessary to expose the class and methods as Web services, as well as providing a contract that clients can use to determine how to call the Web service and the return types to expect from it.

Declaring a Web Service

Web services in ASP.NET are defined in files with the .asmx file extension. The actual code for an ASP.NET Web service can reside either in the .asmx file or in a precompiled class. Either way, the Web service is declared in the .asmx file by adding the `@WebService` directive to the file. The syntax for declaring a Web service in which the class will be defined within the .asmx files is

```
<%@ WebService Language="C#" Class="ClassName" %>
```

ClassName is the name of the class that contains the Web service methods. The syntax for declaring a Web service in which the code that defines the class and methods resides in a compiled class is

```
<%@ WebService Class="NamespaceName.ClassName,AssemblyName" %>
```

NamespaceName is the name of the namespace in which the class for the Web service resides, *ClassName* is the name of the class that contains the Web service methods, and *AssemblyName* is the name of the assembly that contains the compiled code for the Web service. This assembly should reside in the bin directory of the Web application in which the .asmx file resides. The *NamespaceName* portion of the *Class* attribute is optional if you have not specified a namespace for your class, but it is good practice to always specify a namespace for your classes. The *AssemblyName* portion of the *Class* attribute is also optional, since ASP.NET will search the bin directory for the assembly if no assembly name is provided. However, you should always provide an assembly name to avoid the performance impact of doing a search the first time the Web service is accessed.

Note Unlike using code-behind with ASP.NET Web Forms, you cannot use the `src` attribute with a Web service. This means that you must precompile the code-behind class that is to be used as the basis for the Web service, either manually using the command-line compiler or by building a Microsoft Visual Studio .NET Web Service application.

Creating the Web Service Class

Each `.asmx` file is associated with a single class, either inline in the `.asmx` file or in a code-behind class (the default in Visual Studio .NET). Either way, the syntax of a Web service class is as follows:

```
using System.Web.Services namespace NamespaceName public class ClassName :
System.Web.Services.WebService // Methods exposed by the Web service }
```

`ClassName` is the name of the Web service class.

Inheriting from WebService

In this syntax example, an `Imports` statement imports the `System.Web.Services` namespace. This namespace contains the classes and attributes that provide ASP.NET's built-in support for XML-based Web services. In this case, the class we care about is `WebService`, which is inherited in this example using the `Inherits` keyword.

Inheriting from the `WebService` class is not required for a class to function as a Web service, but the `WebService` class provides useful plumbing for Web services. This includes access to `Application`, `Session`, and other ASP.NET objects, including the `Context` object, which provides access to information about the HTTP request that invokes the Web service. Any Web service you create using Visual Studio .NET will use the `WebService` class as the base class.

Using the <WebService(> Attribute to Specify XML Namespace

XML-based Web services use XML namespaces to uniquely identify the endpoint or listener to which a client sends requests. By default, ASP.NET sets the XML namespace for a Web service to `http://tempuri.org/projectname/classname`. If you plan to make your Web service available publicly over the Internet, you might want to change the default namespace to avoid potential conflicts with other XML-based Web services using the same names as your Web service, and which also might use the default namespace. Using a domain name that you control can help ensure uniqueness.

You can modify the default namespace for a Web service class by adding the `WebService` metadata attribute to the class definition and setting its `Namespace` property to the desired value for the new namespace. This value can be any unique URI that you control, such as your organization's domain name. The namespace is used only for identification—the URI specified is not used in any other way, and need not be a valid, accessible Web site.

In Microsoft Visual C# .NET, metadata attributes use the syntax `AttributeName(PropertyName=value)` and are placed on the same line as the entity (class, method, and so on) that they modify:

```
[WebService(Namespace="http://www.aspnetsbs.com/webservices/")] public class ClassName
```


You can set multiple properties in an attribute declaration and separate them by commas. Placing each property definition on a separate line can substantially improve code readability, particularly if you are using many properties in an attribute declaration.

Using the `<WebMethod>` Attribute to Expose Methods

Once you've declared the class that implements the Web service, you will need to add one or more methods that provide functionality for the Web service. Each method will use the *WebMethod* metadata attribute to declare the method as one for which ASP.NET should provide the necessary plumbing to expose it to clients.

To add the *WebMethod* attribute to a Visual C# .NET method, use the following syntax:

```
[WebMethod()] public <ReturnType> MethodName( // method code )
```

MethodName is the name of the method to be exposed by the Web service; *ReturnType* is the type of the value (if any) to be returned by the method. Even if your method does not return a value that is a result of a calculation or other action, it is usually a good idea to at least return a value that indicates the success or failure of the method.

`<WebMethod>` Attribute Properties

Like the *WebService* attribute, the *WebMethod* attribute has properties that you can use to alter the behavior of the Web method. Table 6-1 describes these properties.

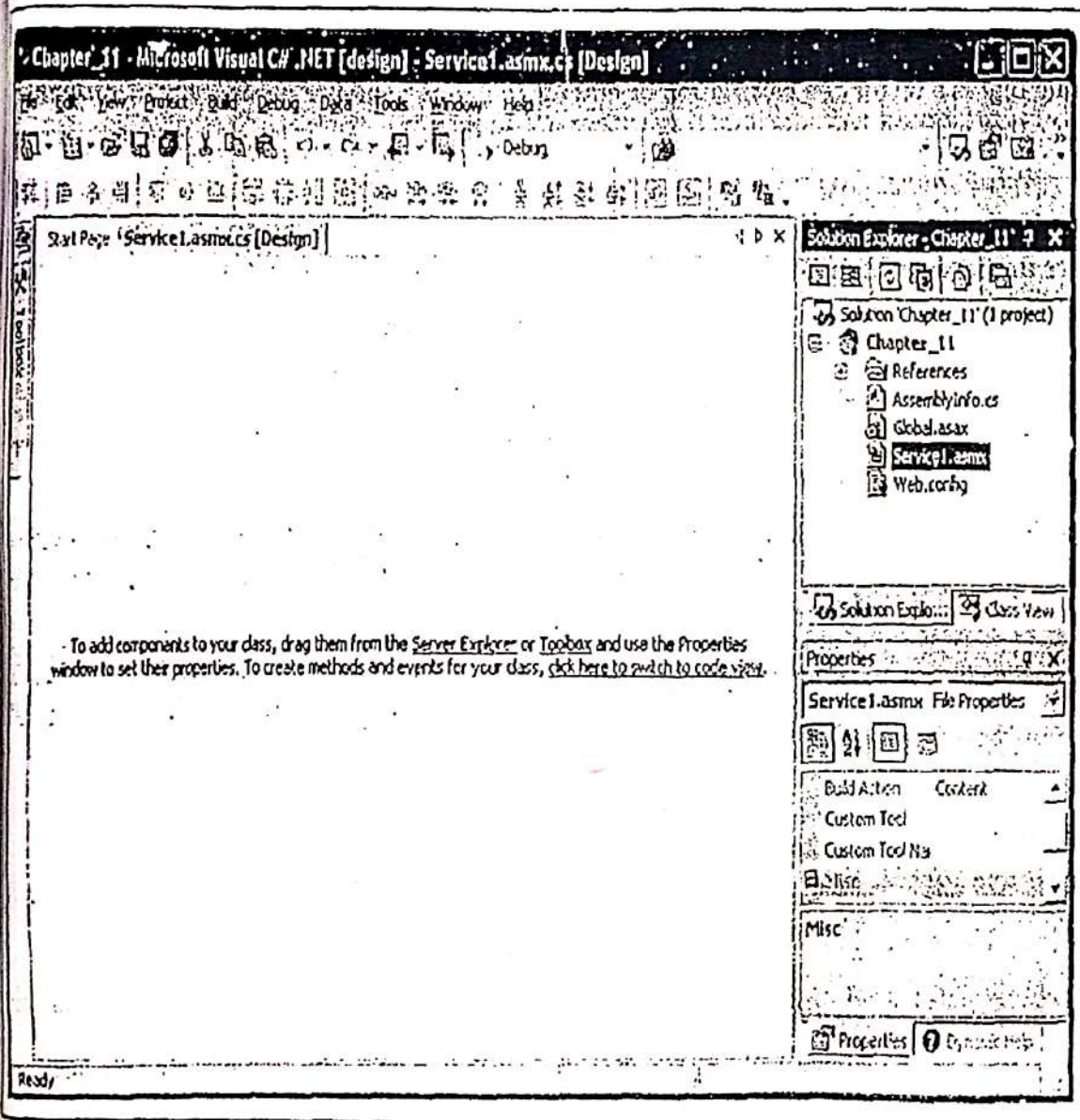
Table 6-1: WebMethod Properties

Property	Description
<i>BufferResponse</i>	Determines whether output from the method is buffered in memory before being sent to the client. The default is <i>True</i> .
<i>CacheDuration</i>	Allows the output of a Web service method to be cached on the server for the specified duration, in seconds. The default is 0, which turns off caching for the method.
<i>Description</i>	Sets a text description of the Web service method. This description is shown in the documentation pages autogenerated by ASP.NET, and it's included in the Web Service Description Language (WSDL) contract for the Web service.
<i>EnableSession</i>	Determines whether session support is enabled for the Web service method. Default is <i>False</i> . Enabling session state requires the Web service class to inherit from <i>WebService</i> . Note that enabling session state for a Web service can have a negative impact on performance.
<i>MessageName</i>	Allows methods to be given alias names. This can be useful when you have an overloaded method, in which the same method name has multiple implementations that accept different input parameters. In this case, you can supply a different <i>MessageName</i> for each overloaded method version, allowing you to uniquely identify each one. The default is the method name.

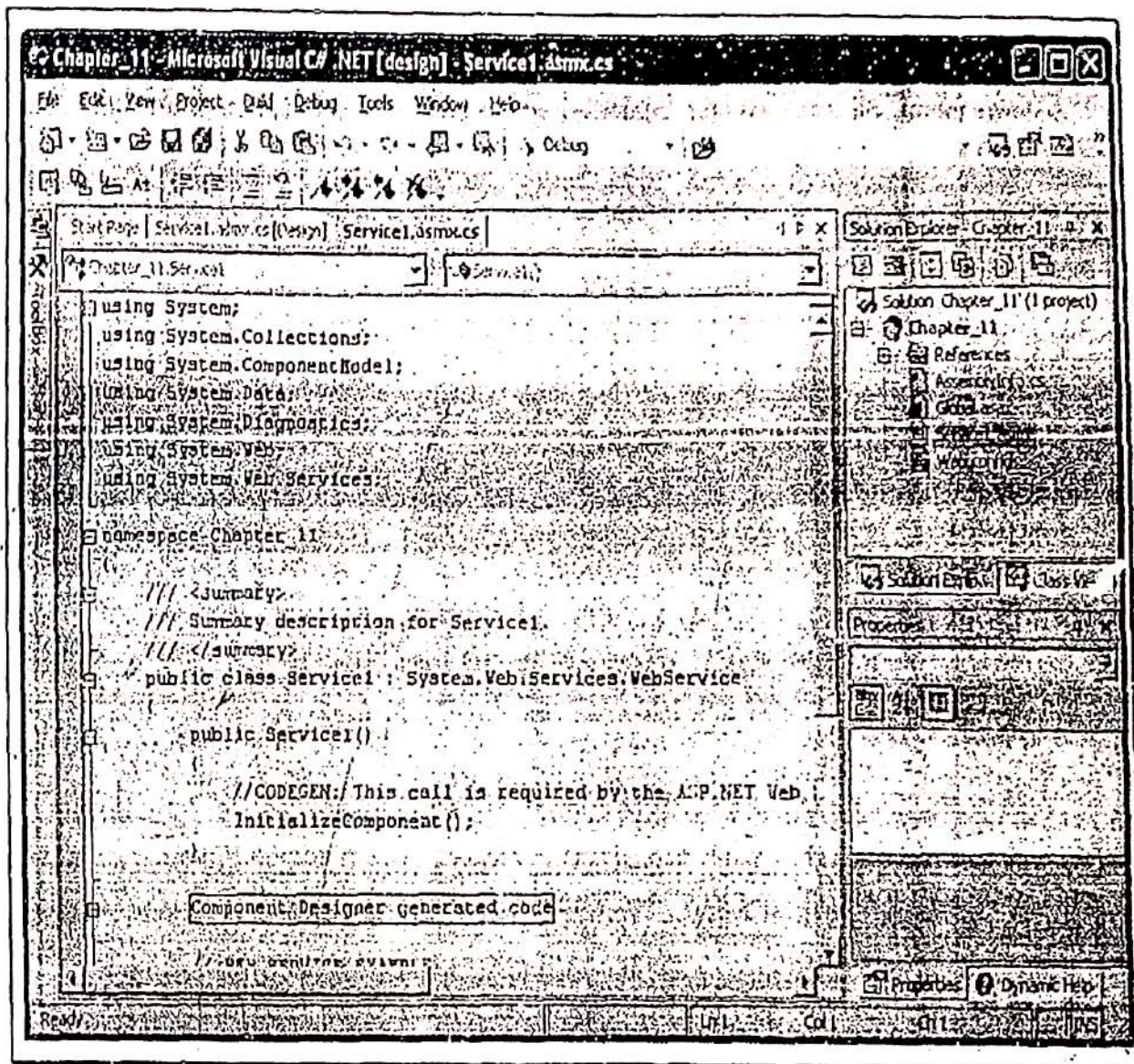
TransactionOption	Allows adding support for COM+ transactions in a Web service method. Allowable values are <i>Disabled</i> , <i>NotSupported</i> , <i>Supported</i> , <i>Required</i> , and <i>RequiresNew</i> . The default is <i>Disabled</i> . Note that this property requires the use of an <i>Assembly</i> directive to load the <i>System.EnterpriseServices</i> assembly into the Web service application.
-------------------	---

Create a HelloWorld Web service

1. Create a new project in Visual Studio .NET. Unlike previous projects, this project should be an ASP.NET Web Service rather than an ASP.NET Web Application. Type the name of the project as **Chapter_11**, and click OK. The resulting screen should show the default Web Service, **Service1.aspx.cs**, in design view, as shown in the following illustration.



2. Switch to code view. You can click on the link provided in design view to switch to code view, or use the standard Visual Studio .NET key, F7. However you switch to code view, the screen should look similar to the following illustration:



Unlike Web Applications, in which a great deal of the work of building the page is done in design view, Web services are generally modified using code directly.

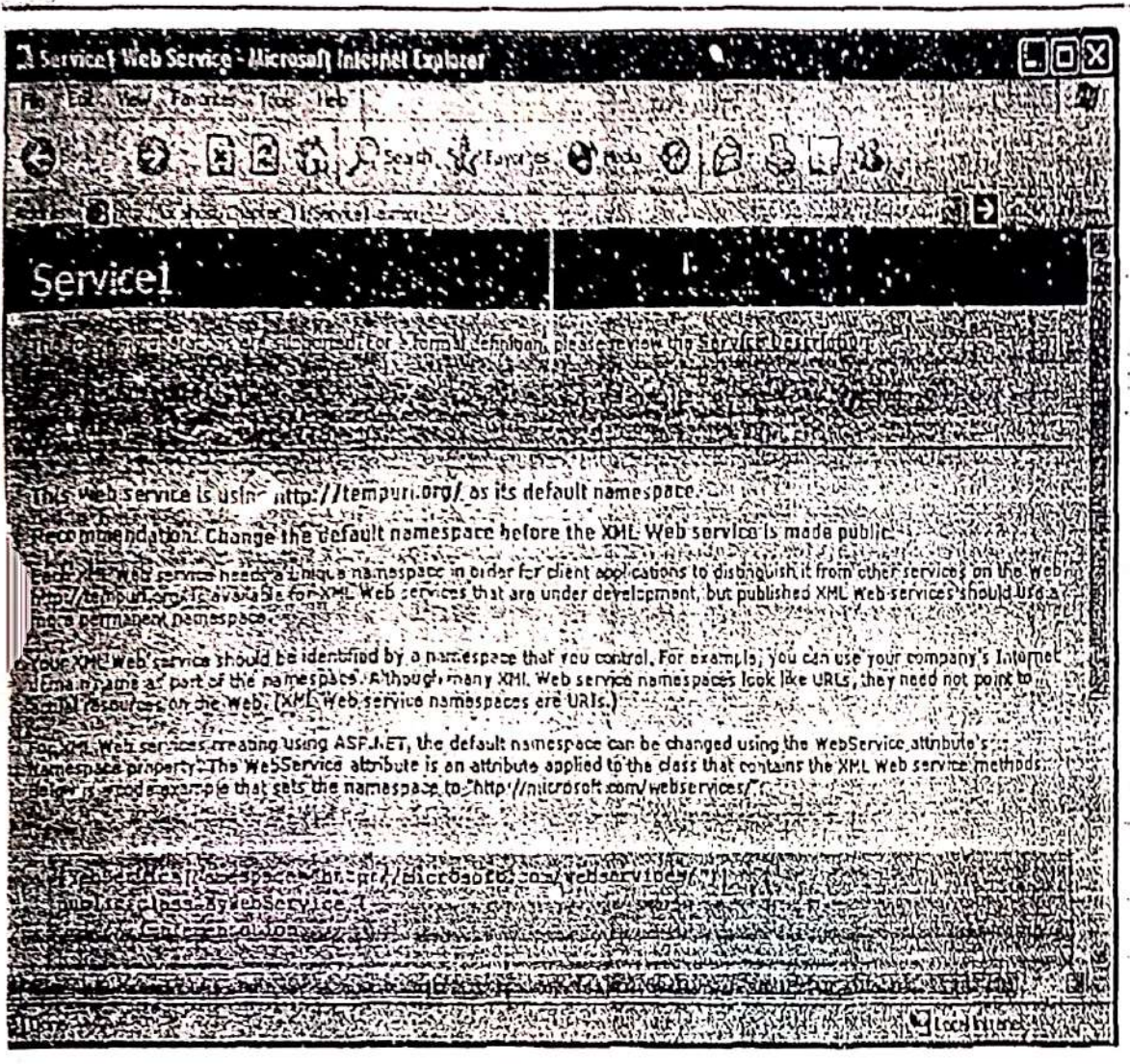
3. Toward the bottom of the file, locate a comment titled WEB SERVICE EXAMPLE. The code will look similar to the following:

```
// WEB SERVICE EXAMPLE // The HelloWorld() example service returns the string Hello
// World // To build, uncomment the following lines then save and build the // project // To
// test this web service, press F // [WebMethod // public string HelloWorld( // //
// return "Hello World" // }
```

4. Remove the comments from the beginning of each line from [WebMethod] to the closing curly brace for the function. Rather than leave this example exactly as it is created by Visual Studio .NET, add a parameter to HelloWorld, and append the parameter to the value returned. The finished method should look like the following code:

```
[WebMethod] public string HelloWorld(string name) { return "Hello World, from " + name; }
```


5. Save the file, and then build the project.
6. Right-click on Service1.asmx in Solution Explorer. Select Browse With from the drop-down menu. Select Microsoft Internet Explorer from the browser list, and then click Browse. The resulting screen should look similar to the following illustration.

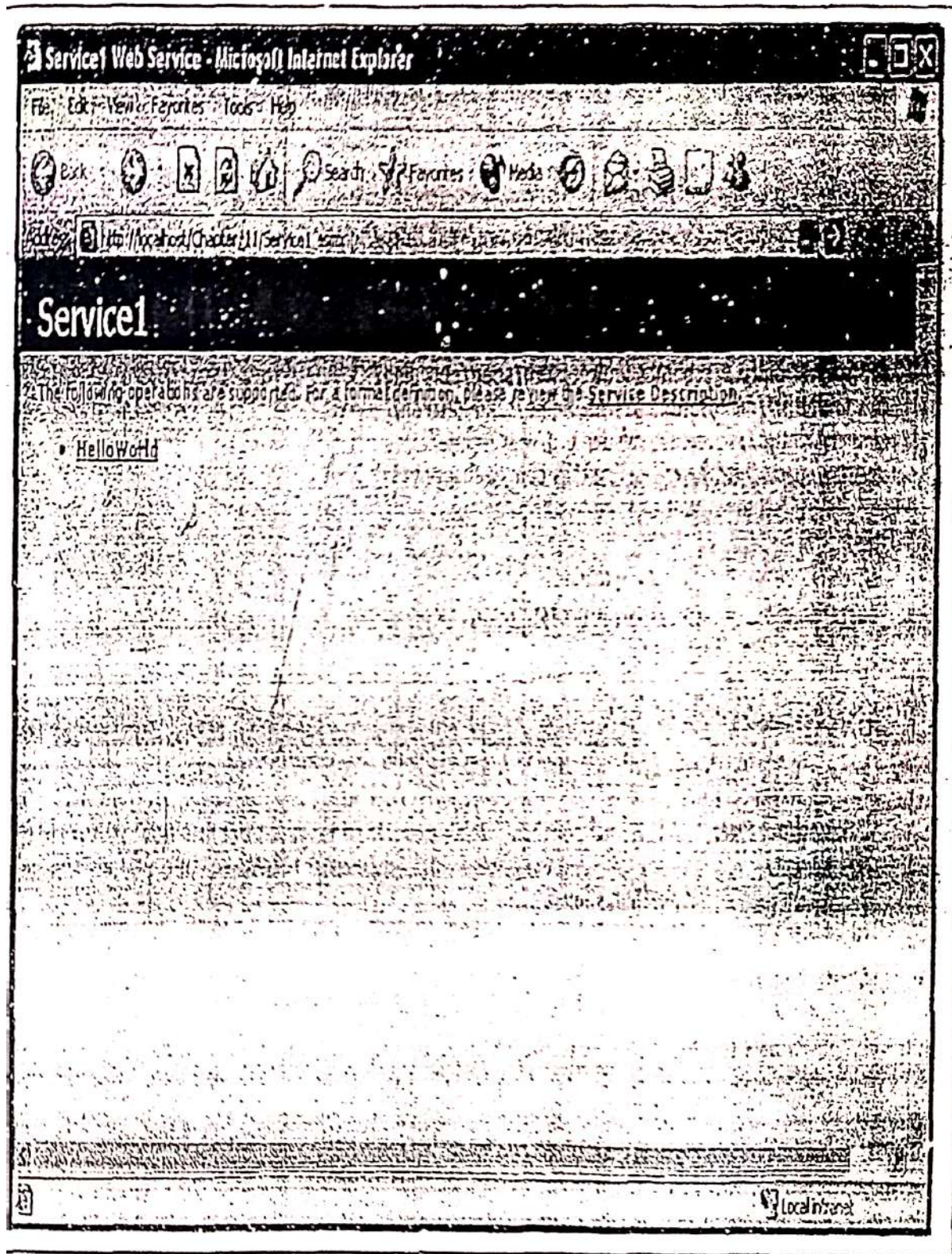


One thing that stands out on the illustration is the warning that the Web service is using `http://tempuri.org/namespace`. There is a recommendation to change the namespace before Web service is made public. Although it is unlikely this Web service will ever be made public, fix this now. Close the browser and return to the source code.

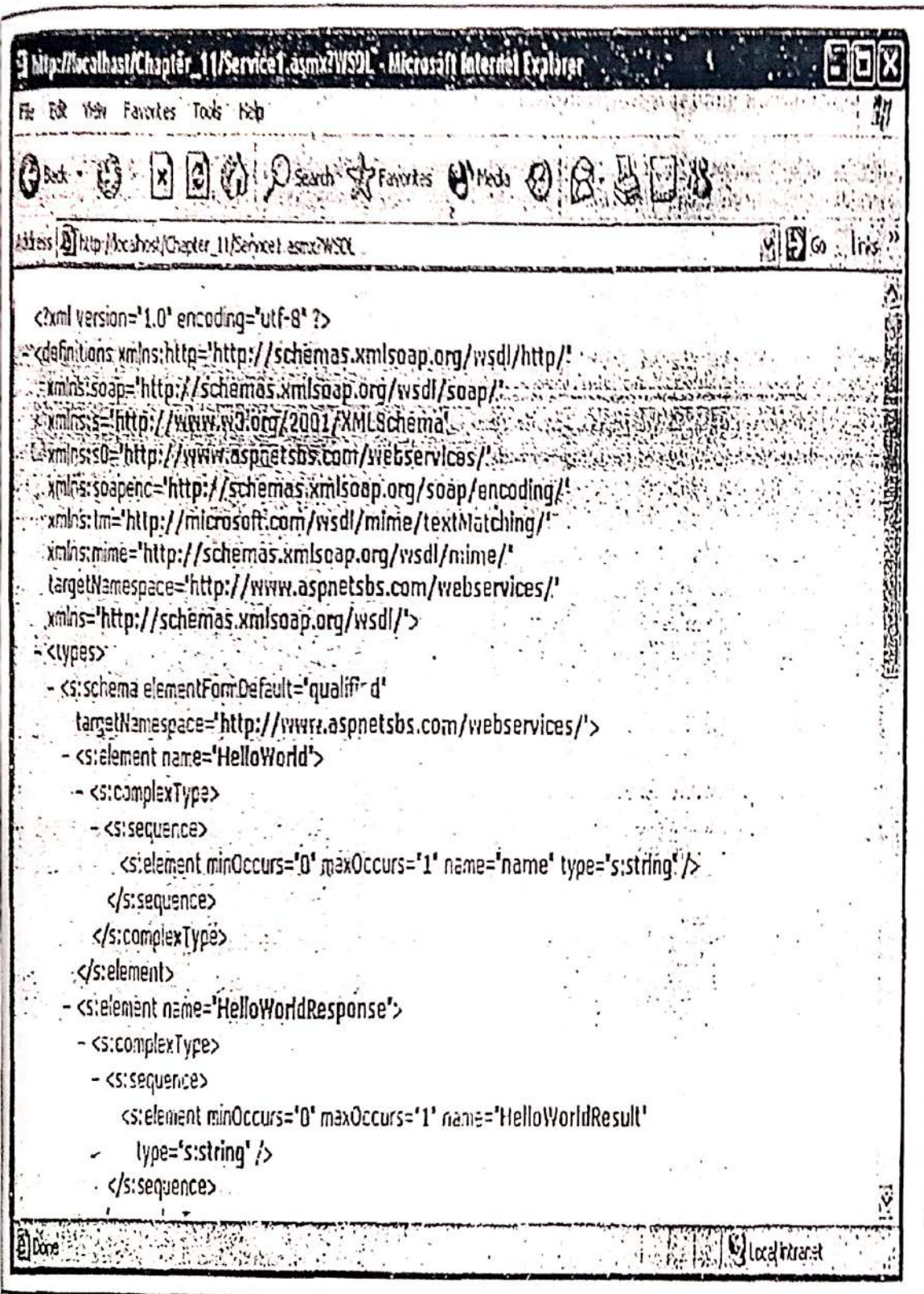
7. Add a line directly below the closing summary tag just above the top of the class declaration. Note that if you press Enter at the end of the line with the closing summary tag, you should see that Visual Studio .NET will insert three slashes (as a convenience, presuming you are adding more comments). Delete the three slashes if they appear. On the line just below the closing summary tag, add the following (you can substitute a different namespace value, if you like, as long as it is unique):

```
[WebService(Namespace="http://www.aspnetbs.com/webservices/")]
```

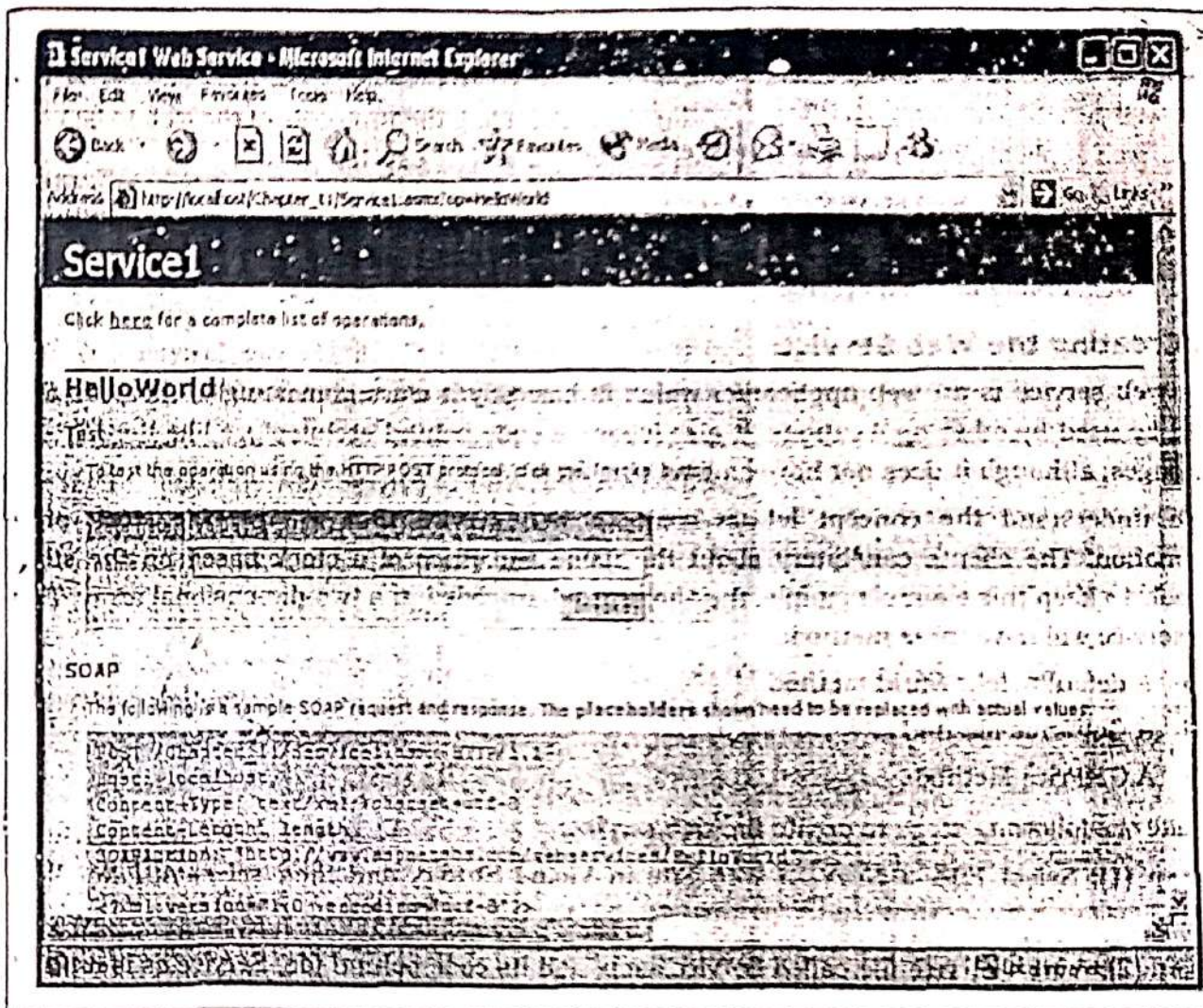

8. Save the open files, and build the project again.
9. Browse the web service again using Internet Explorer, as described in step 6. The result should look similar to the following illustration:



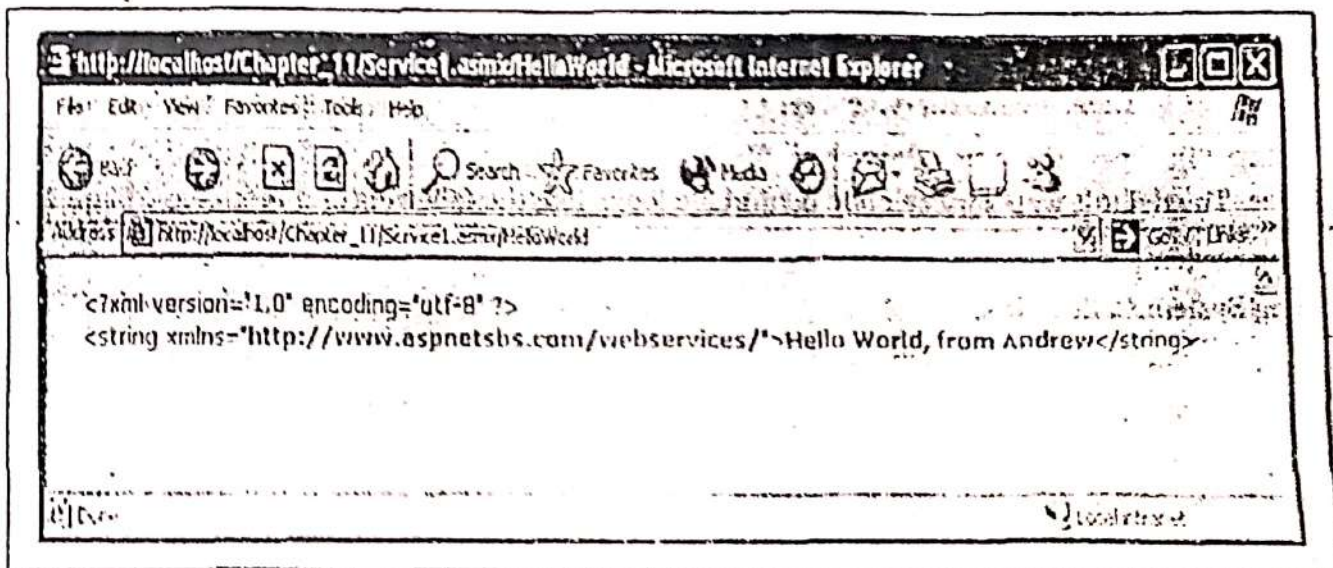
There are two links on this page. The first is Service Description. Click this link, and a screen similar to the following illustration will appear:



10. You can scroll through the entire document and see the complete Web Services Description Language (WSDL) contract for the Web service method. Click the Back button on the browser and select the other link on the page, *HelloWorld*. The screen should look similar to the following illustration



11. Enter your name in the text box provided for the name, and then click the Invoke button. The resulting screen should look similar to the following illustration, which shows the XML output returned by the Web service.



6.7 DEPLOYING WEB SERVICE

A web service is a web-based functionality accessed using the protocols of the web to be used by the web applications. There are three aspects of web service development:

- (I) Creating the web service
- (II) Creating a proxy
- (III) Consuming the web service

(I) Creating the Web Service

A web service is an web application which is basically a class consisting of methods that could be used by other applications. It also follows a code-behind architecture like the ASP.Net web pages, although it does not have an user interface.

To understand the concept let us create a web service that will provide stock price information. The clients can query about the name and price of a stock based on the stock symbol. To keep this example simple, the values are hardcoded in a two-dimensional array. This web service will have three methods:

- A default HelloWorld method
- A GetName Method
- A GetPrice Method

Take the following steps to create the web service:

Step (1): Select File--> New --> Web Site in Visual Studio, and then select ASP.Net Web Service.

Step (2): A web service file called Service.asmx and its code behind file, Service.cs is created in the App_Code directory of the project.

Step (3): Change the names of the files to StockService.asmx and StockService.cs.

Step (4): The .asmx file has simply a WebService directive on it:

```
<%@ WebService Language="C#"
    CodeBehind="~/App_Code/StockService.cs"
    Class="StockService" %>
```

Step (5): Open the StockService.cs file, the code generated in it is the basic Hello World service. The default web service code behind file looks like the following:

```
using System;
using System.Collections;
using System.ComponentModel;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.Services;
```



```

using System.Web.Services.Protocols;
using System.Xml.Linq;
namespace StockService
{
    /// <summary>
    /// Summary description for Service1
    /// </summary>
    [WebService(Namespace = "http://tempuri.org/")]
    [WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
    [ToolboxItem(false)]
    // To allow this Web Service to be called from script,
    // using ASP.NET AJAX, uncomment the following line.
    // [System.Web.Script.Services.ScriptService]
    public class Service1 : System.Web.Services.WebService
    {
        [WebMethod]
        public string HelloWorld()
        {
            return "Hello World";
        }
    }
}

```

Step (6): Change the code behind file to add the two dimensional array of strings for stock symbol, name and price and two web methods for getting the stock information.

```

using System;
using System.Linq;
using System.Web;
using System.Web.Services;
using System.Web.Services.Protocols;
using System.Xml.Linq;

[WebService(Namespace = "http://tempuri.org/")]
[WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
// To allow this Web Service to be called from script,
// using ASP.NET AJAX, uncomment the following line.
// [System.Web.Script.Services.ScriptService]
public class StockService : System.Web.Services.WebService

```



```

{
    public StockService () {
        //Uncomment the following if using designed components
        //InitializeComponent();
    }

    string[,] stocks =
    {
        {"RELIND", "Reliance Industries", "1060.15"},
        {"ICICI", "ICICI Bank", "911.55"},
        {"JSW", "JSW Steel", "1201.25"},
        {"WIPRO", "Wipro Limited", "1194.65"},
        {"SATYAM", "Satyam Computers", "91.10"}
    };

    [WebMethod]
    public string HelloWorld() {
        return "Hello World";
    }

    [WebMethod]
    public double GetPrice(string symbol)
    {
        //it takes the symbol as parameter and returns price
        for (int i = 0; i < stocks.GetLength(0); i++)
        {
            if (String.Compare(symbol, stocks[i, 0], true) == 0)
                return Convert.ToDouble(stocks[i, 2]);
        }
        return 0;
    }

    [WebMethod]
    public string GetName(string symbol)
    {
        // It takes the symbol as parameter and
        // returns name of the stock
        for (int i = 0; i < stocks.GetLength(0); i++)
        {
            if (String.Compare(symbol, stocks[i, 0], true) == 0)
                return stocks[i, 1];
        }
    }
}

```

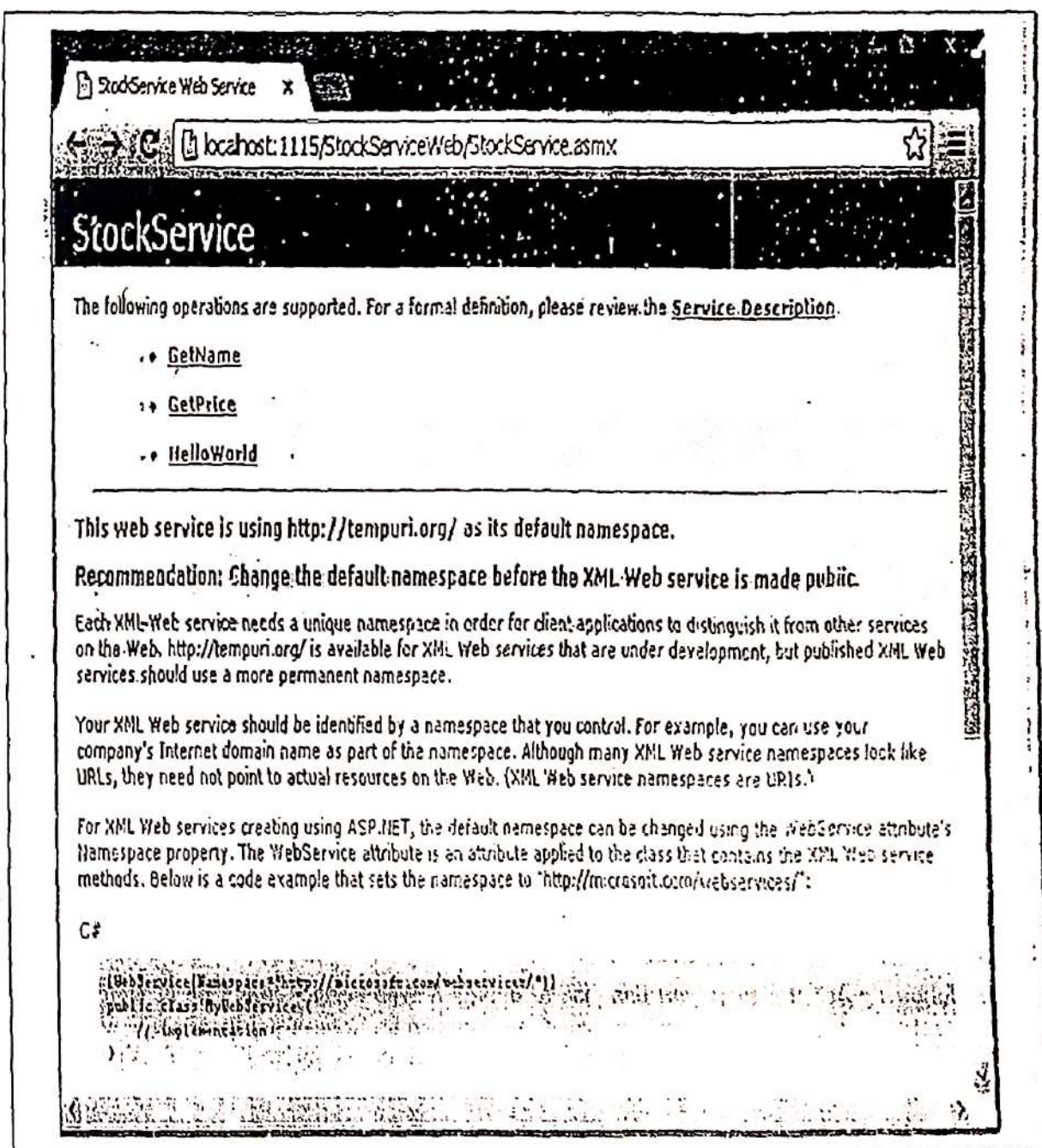


```

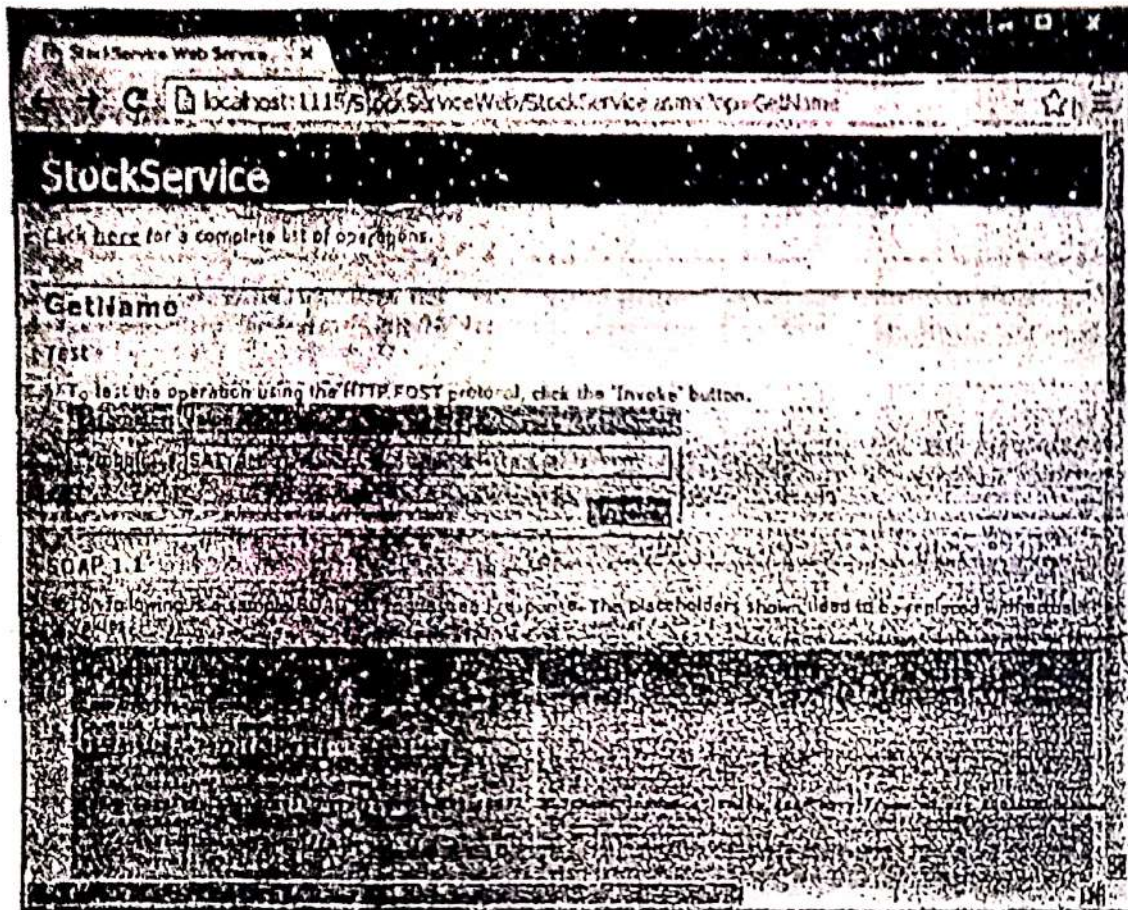
-- }
return "Stock Not Found";
}
}

```

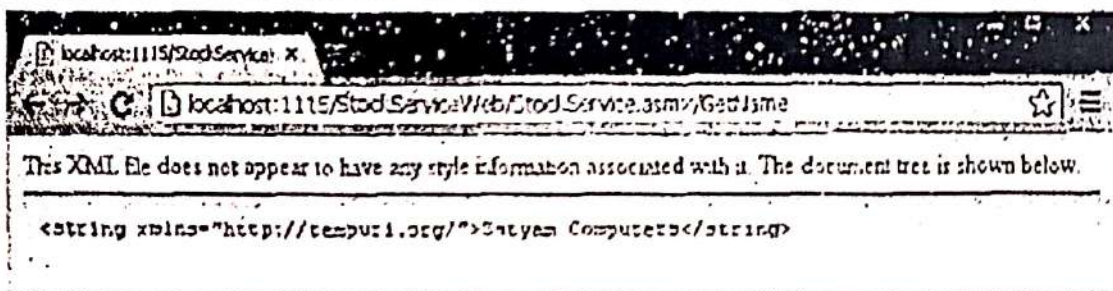
Step (7) : Running the web service application gives a web service test page, which allows testing the service methods.



Step (8): Click on a method name, and check whether it runs properly.



Step (9): For testing the GetName method, provide one of the stock symbols, which are hardcoded, it returns the name of the stock



(II) Creating a proxy: See Topic 6.10

(III) Consuming the web service: See Topic 6.10

7.1 Web service deployment tools

The following are the tools you can use to deploy a Web service, any one of which may be more appropriate depending on the complexity and circumstances of any particular Web service project:

- Visual Studio Web Setup Project
- Visual Studio Project Copy
- DOS XCOPY command

Deployment using a Web Setup Project

If you have built your Web service using Visual Studio, the IDE provides a Web Setup Project template that utilizes the services of the Microsoft Windows Installer technology to create a deployment package for your Web service. A Web Setup Project in Visual Studio creates a MSI file (also called an installation package) that, when executed, creates and configures a virtual directory on the Web server, copies the files required to execute the Web service to the virtual directory, and registers any additional assemblies needed by the Web service.

One of the advantages of using a Web Setup Project is that the installation package automatically handles any registration and configuration issues that your Web service may depend upon, relieving you of this burden. In general, the basic steps required to deploy a Web service using the Web Setup Project method are as follows:

1. Create a Web Setup Project using the Web Setup Project template in Visual Studio.
2. Build the project.
3. Copy the installation package to the target Web server.
4. Run the installation package on the target Web server.

(Note: You must have administrative privileges on the target Web server computer in order to successfully install the Web service using the installation package.)

It is important to remember the following facts about the setup packages created by Web Setup Projects:

- The user has the opportunity to specify an alternate virtual directory target during the setup process.
- The setup process creates a new virtual directory and configures the virtual directory for the Web service.

In summary, although the Web Setup Project method of deployment requires more upfront work to create and configure properly, the result is a setup package that provides a solid, repeatable, and reliable installation experience that guarantees that your Web service will operate correctly.

Deployment using Project Copy

As an alternative to using a Web Setup Project to deploy a Web service built using Visual Studio, you can use the Project Copy method, because it is a simpler deployment method than using a Web Setup Project. However, copying your Web service project files does not perform such tasks as virtual directory configuration or file registrations that may be necessary for your Web service to function correctly. In more complex scenarios, the Web Setup Project method described in the previous section is a superior and more reliable choice (although somewhat more complex to set up and configure). Be that as it may, simple Web services can be easily and successfully deployed to target Web servers with very little effort using the Project Copy feature in Visual Studio. The following are the three options available for copying files:

- **Only Files Needed To Run This Application:** Copies all DLLs with references in the /bin folder as well as any files marked with a BuildAction of Content.
- **All Project Files:** Copies all project files created and managed by Visual Studio.

- **All Files In The Source Project Folder:** Copies all Visual Studio project files as well as other files that reside in the project folders. The default option is to copy only the files needed to run the application.

Deployment using XCOPY

The DOS XCOPY command is perhaps the simplest method for deploying a Web service to a target Web server. However, as is the case with the Visual Studio Copy Project feature, XCOPY simply copies files from one location to another. It does not create or configure virtual directories for your Web service, nor does it register or configure any dependent assemblies outside of the .NET Framework. You can type XCOPY /? at a command prompt to get help on the XCOPY command-line syntax and available options.

6.2 PUBLISHING WEB SERVICES

Publishing a Web service enables potential consumers to locate and interrogate service descriptions that instruct the consumer on how to interact with the Web service. The process of locating and interrogating Web service descriptions is referred to as the discovery process. The following are the two methods for enabling discovery of a Web service:

- DISCO
- UDDI

You may choose to use one or both of these methods based on the consumer audience you are trying to reach.

If your consumer population is fairly small (or well known), you could simply point them to the target Web server and deploy the DISCO file on this server. In this case, the consumers will invoke the discovery process against the URL of the target server and locate your Web service description. In this situation, you only need to deploy your DISCO documents to the proper server and inform the consumers of the URL to the server.

On the other hand, if your consumer population is relatively large or unknown (in which case it is impractical to provide them with a pointer to the target Web server where the service is located), you will need to provide a mechanism for the consumers to find where your DISCO and/or Web service descriptions are located, just as Web users utilize search engines to find Web pages. In this situation, you will need to publish your Web service through UDDI.

The DISCO document and the UDDI business registry provide the mechanisms to solve these issues. Let's explore how to use these technologies to enable potential consumers to locate the essential information they need in order to use your Web service.

Publishing with DISCO

Consumers of Web services enact a discovery process to locate Web services. The discovery process searches for XML-encoded discovery documents that contain pointers to other resources that describe the Web service. Encoding discovery documents in XML enables tools such as Visual Studio to programmatically discover the availability of Web services. This is how the Web Reference metaphor works in Visual Studio when you provide it with a specific discovery URL.

For those who don't use Visual Studio, they can use the disco.exe tool that comes with the .NET Framework SDK to discover Web services.

Web service discovery via the Web Reference feature in Visual Studio or the disco.exe tool in the .NET Framework SDK is useful when a consumer knows the URL to the server hosting the Web service or the application virtual directory. Visual Studio automatically creates a DISCO file when you create a Web service project. This file has an extension of .vsdisco and is stored in the main application virtual directory (along with the ASMX file). This file contains links to the resources that describe the Web service. If you are not using Visual Studio to create your Web service (or find them), you need to manually create the DISCO document. In this case, you must create an XML file containing DISCO elements that can be used to find your Web service description documents. The following is a sample DISCO document for our CTemp Web service:

```
<?xml version="1.0" ?>
  <disco:discovery xmlns:disco="http://schemas.xmlsoap.org/disco/">
    <xmlns:scl="http://schemas.xmlsoap.org/disco/scl">
      <scl:contractRef ref="http://jdc7200cte/Services/CTemp/CTemp.asmx?WSDL"/>
      <disco:discoveryRef ref="SomeFolder/default.disco" />
    </disco:discovery>
```

The DISCO document consists of a discovery element that serves as a container for contractRef and discoveryRef elements. You can specify as many contractRef and discoveryRef elements as you desire. This makes it possible to provide information for more than one Web service from within a single DISCO document.

The contractRef element is used to provide a pointer to the WSDL document that describes the message formats and exchanges supported by the Web service. In the preceding example, you can see that a single contractRef element has been included that provides a pointer to the WSDL document for the CTemp Web service. The contractRef element is optional. References specified by the ref attribute can be absolute or relative. If you specify a relative reference, the reference is relative to the folder in which the DISCO document resides.

The discoveryRef element is used to provide a pointer to other DISCO documents. This allows you to logically link the discovery process to several Web services, which may have independent DISCO documents. In the preceding example, a single discoveryRef element has been included that provides a pointer to another DISCO document. The discoveryRef element is optional.

Because a DISCO document simply contains pointers to the resources that describe a Web service, you can physically deploy your DISCO documents anywhere you want (in other words, they don't have to be physically deployed with the Web services that they describe). This provides you with the capability to enable discovery of any set of Web services that may be physically distributed on your network, but allow users to browse to a single point at which to begin the discovery process to find them.

To make it easier for consumers to find your Web service, you can create a DISCO file in the root directory of your Web server named Default.disco, which then links to the various Web service DISCO files that you have implemented. That way, a consumer can simply browse to the root directory of the server to begin the discovery process using tools such as disco.exe.

Publishing with UDDI

UDDI stands for Universal Description, Discovery and Integration is an XML-based standard describing, publishing, and finding Web services. It has following features:

- UDDI is a specification for a distributed registry of Web services.
- UDDI is platform independent, open framework.
- UDDI can communicate via SOAP, CORBA, Java RMI Protocol.
- UDDI uses WSDL to describe interfaces to web services.
- UDDI is seen with SOAP and WSDL as one of the three foundation standards of web services.
- UDDI is an open industry initiative enabling businesses to discover each other and define how they interact over the Internet.

UDDI has two parts:

- A registry of all a web service's metadata including a pointer to the WSDL (Web Services Description Language) description of a service
- A set of WSDL port type definitions for manipulating and searching that registry

Why UDDI? What is the need for such a registry? As we look toward a software landscape of thousands—perhaps millions—of Web Services, some tough challenges emerge:

- How are Web Services discovered?
- How is this information categorized in a meaningful way?
- What implications are there for localization?
- What implications are there around proprietary technologies? How can I guarantee interoperability in the discovery mechanism?
- How can I interact with such a discovery mechanism at run time once my application is dependent upon a Web Service?

In response to these challenges, the UDDI initiative emerged. A number of companies, including Microsoft, IBM, Sun, Oracle, Compaq, Hewlett Packard, Intel, SAP, and over three hundred other companies, came together to develop a specification based on open standards and non-proprietary technologies to solve these challenges.

UDDI describes a registry of Web services and programmatic interfaces for publishing, retrieving, and managing information about services described therein. In fact, UDDI itself is of a Web services. The UDDI specification defines services that support the description and discovery of (1) businesses, organizations, and other Web services providers, (2) the Web services they make available, and (3) the technical interfaces which may be used to access and manage these services. UDDI is based upon several other established industry standards, including HTTP, XML, XML Schema (XSD), SOAP, and WSDL.

Discovery of Web services via DISCO is sufficient if you know the URL to the Web server or application virtual directory that hosts the Web service. However, in cases where this information is unavailable, you must use a more generalized search tool Universal Description.

Discovery, and Integration (UDDI) enable Web service consumers to search for and locate Web services, even if the consumer is unaware of the exact location, owner, or author of the Web service.

How Does It Work?

The UDDI data is hosted by operator nodes, companies that have committed to running a public node that conforms to the specification governed by the UDDI.org consortium. Today, two public nodes exist that conform to the Version 1 UDDI specification: One is hosted by Microsoft and one by IBM. Hewlett-Packard has committed to hosting a node under the Version 2 specification as well. Host operators are required to replicate data between one another across a secure channel, providing a redundancy to the entire UDDI cloud. Data can be published to one node and, after replication, can be discovered on another node. Today, replication occurs at 24-hour intervals; in the future, as more applications are dependent on the UDDI data, the intervals will become shorter between replication.

It is worth noting that there are no proprietary requirements as far as how a host operator implements its node. The node simply must conform to the UDDI specification. The Microsoft node (<http://uddi.microsoft.com/default.aspx>), for example, has been written entirely in C# and runs in production on the .NET Beta 2 Common Language Runtime. The code base takes significant advantage of the native SOAP support and serialization offered by .NET system classes. On the backend, the Microsoft operator node utilizes Microsoft® SQL Server 2000 as its data store. Suffice to say that the IBM node is using different technologies to run its node. However, the two nodes behave identically, because they conform to the same set of SOAP-based XML API calls. Client tools can interoperate with the nodes seamlessly. As such, the UDDI public cloud serves as a prime example of how the XML Web Services model works across heterogeneous environments.

Taking a look at what data is stored in UDDI and how it is structured is the next step in understanding the UDDI initiative. UDDI is relatively lightweight; it is designed as a *registry*, not a *repository*. The distinction is subtle, but crucial. A registry redirects a user to resources, whereas a repository is an actual information store. Consider the Microsoft® Windows® registry as an example: It contains basic settings and parameters, but ultimately leads an application to a resource or binary. Looking up a COM component based upon its Prog ID leads to a Class ID, which leads to the location of the binary itself.

UDDI behaves similarly: Like the Windows registry, it relies on Globally Unique Identifiers (GUID) to guarantee the ability to perform look-ups and determine the location of resources. UDDI queries ultimately lead to an interface—a .WSDL file, .XSD file, .DTD file, and so on—or an implementation (such as an .ASMX or .ASP file) located on another server. UDDI can thus answer the following kinds of questions:

- "What Web Service interfaces have been published that are based on WSDL and established for a given industry?"
- "What companies have written an implementation around one of these interfaces?"
- "What Web Services, categorized in a certain way, are offered today?"
- "What Web Services does a given company offer?"

- "Who do I need to contact about using a company's Web Service?"
- "What are the implementation details of a particular Web Service?"

6.2 FINDING WEB SERVICES

When using the .NET tools for locating Web services, the tool you use depends on how much you know about the location of the Web service before you start.

Web service directories provide a centralized, Internet-accessible location that consumers can use to find Web services that have been offered for use by other companies or organizations. You can think of Web service directories as a type of Web portal or "Yellow Pages" specifically suited for listing and locating Web services.

Using Web service directories, you can search for Web services using a variety of structured criteria, such as business type, industry, type of goods produced, services offered, and so on. For example, if you were looking for a credit card validation Web service, you could search the directory using personal credit companies as the criteria. Currently, the Universal Description, Discovery, and Integration (UDDI) specification is the de facto standard for cataloging and finding Web services. The UDDI organization (located on the Web at <http://www.uddi.org>), composed of several hundred industry participants, has created a directory schema, distributed repository, and APIs for manipulating and querying the repository.

Currently, Microsoft and IBM both have cooperating UDDI directories operational and available for general use. These sites include operational Web services, which can be called to programmatically manipulate and query the UDDI registry database. If you are using Microsoft Visual Studio .NET to create your Web services or Web service consumer applications, the Web Reference feature of Visual Studio has the ability to search these online UDDI directories automatically. In fact, the Visual Studio .NET Web Reference feature is itself a consumer of the UDDI Web services.

Alternatively, if you are not using Visual Studio .NET, you can use the UDDI Web site to search for Web services. The Web site contains interactive forms for manipulating and querying the registry database.

If neither of these methods suits your needs, you can also download a UDDI SDK from either Microsoft or IBM, which you can use to create your own custom search tool. We will examine in detail how to use these services in next topic, "Consuming Web Services."

Finding a Web service is essentially the mirror image of advertising a Web service, so the same techniques apply here. There are two techniques for locating a Web service in ASP.NET: discovery documents and UDDI.

(I) Locating a Web Service with a Discovery Document

To use a discovery document to locate Web services, you need the URL for the discovery document that describes the services in which you're interested. You might get the URL for the document from a listing on an organization's public Web site, from an e-mail promotion, or from a catalog.

Once you have the URL for a discovery document, you can use the `disco.exe` command-line utility to generate WSDL contracts for using the Web services described in the discovery document. The syntax for the `disco.exe` utility is

`disco [options] URL`

Where URL specifies the HTTP address of the target Web server that you wish to search for discovery (files of type `.disco`) documents. Here, options represent one or more command-line options for the utility and URL represents the URL to the discovery document. Table 6.1 shows options for the `disco.exe` utility.

Table 6.1: disco.exe Options.

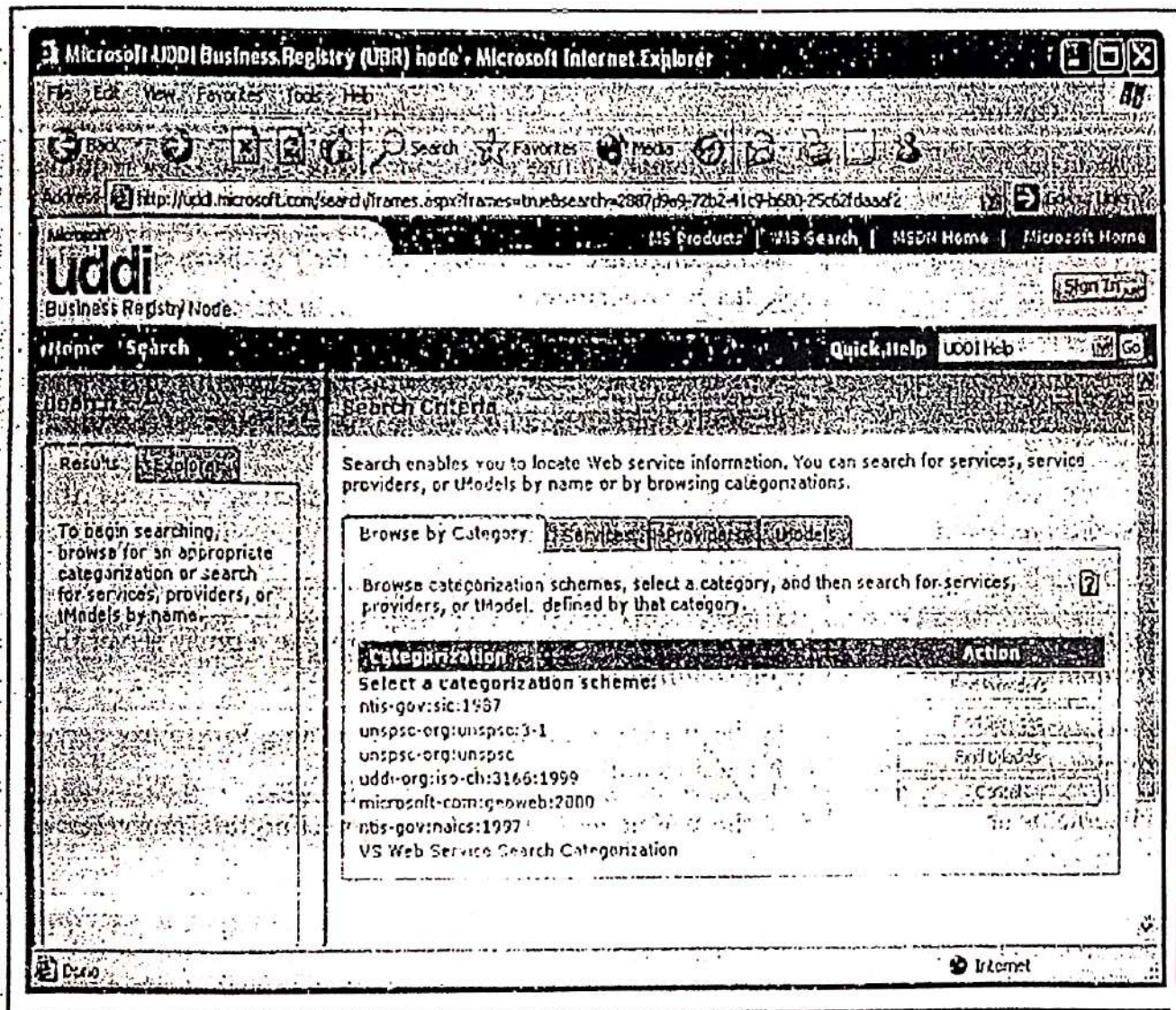
Option	Description
<code>/d:domain</code> or <code>/domain:domain</code>	Sets the domain to use when connecting to a site that requires authentication.
<code>/nosave</code>	Prevents <code>disco.exe</code> from saving files for the discovered WSDL and discovery documents.
<code>/nologo</code>	Prevents the start-up banner from being displayed.
<code>/o:directory</code> or <code>/out:directory</code>	Provides the directory name where output should be saved.
<code>/p:password</code> or <code>/password:password</code>	Sets the password to use when connecting to a site that requires authentication.
<code>/proxy:url</code>	Sets the proxy to use for HTTP requests.
<code>/pd:domain</code> or <code>/proxydomain:domain</code>	Sets the domain to use when connecting to a proxy server that requires authentication.
<code>/pp:password</code> or <code>/proxypassword:password</code>	Sets the password to use when connecting to a proxy server that requires authentication.
<code>/pu:username</code> or <code>/proxyusername:username</code>	Sets the username to use when connecting to a proxy server that requires authentication.
<code>/u:username</code> or <code>/username:username</code>	Sets the username to use when connecting to a site that requires authentication.
<code>/?</code>	Provides help for command-line options.

(II) Locating a Web Service with UDDI

Unlike using a discovery document to locate a Web service, UDDI does not require you to know in advance the URL of a document describing the desired Web service. Any provider of a UDDI business registry usually will provide a tool for searching their instance of the registry for businesses or Web services that meet your needs. The criteria you can search on will depend on the organization hosting that instance. For example, Microsoft's UDDI registry at <http://uddi.microsoft.com> supports searching by business name, business location, discovery

URL, keyword, and a variety of other criteria. The following illustration shows the list of available criteria for searching the Microsoft UDDI registry.

UDDI is a more generalized search tool for locating Web services. Using UDDI, you can specify search terms that enable you to pinpoint Web services you wish to use based on company, industry, service type, and many other classifications. Using these more generalized search terms usually results in the listing of specific URLs to which you can locate information about specific Web services, including the WSDL document that you are ultimately searching for.



Although a simple and effective solution for finding Web services, you may encounter some cases in which you cannot use Visual Studio's Web Reference feature, such as when the Web service is not accessible from the machine on which you are using Visual Studio. Under these circumstances, you can use the WSDL (Web Service Description Language).

Understanding WSDL Files

WSDL is an XML-based standard for documents that describe the services exposed by a Web service. You can view the WSDL contract for any of the Web services you create with ASP.NET by calling up the URL for a Web service, adding `?wsdl` to it.

WSDL contracts are similar to COM type libraries or .NET assembly metadata in that they are used to describe the public members of the Web service, including all publicly available methods, the parameters (and data types) those methods expect when called, and the data type of any return value. Because this information is encoded in a standard XML-based format, any client that can read XML and can understand WSDL and SOAP should be able to effectively consume the Web application that the WSDL contract describes.

The .NET Framework provides the `wsdl` tool for parsing Web service description files. But, in addition to interrogating these files, the `wsdl` tool is able to generate a proxy class, which provides an object-oriented interface for calling methods on the Web service and returning the results to the caller.

Visual Studio automatically creates Web service proxy classes for you when you use the Add Web Reference dialog box to create a reference to a Web service. If, however, you are unable to use Visual Studio's Web Reference feature, you can create the proxy class manually using the `wsdl` tool.

Creating a proxy class with the WSDL tool

The .NET Framework provides a tool named `wsdl.exe` to parse Web service descriptions and generate proxy classes, which can be used by a consumer to call methods on a Web service. The `wsdl` tool is capable of generating proxy classes given any one of the following types of files as input:

- WSDL files
- XSD (XML Schema Definition) files
- DISCO files
- DISCOMAP files

Note that these files are all outputs of .NET's Disco tool. By default, the `wsdl` tool is located in the same place as the Disco tool:

Program Files\Microsoft.NET\FrameworkSDK\Bin

The `wsdl` tool is a console application, so you will need to start it from within a command window just like the Disco tool. You may want to add the path to the .NET Framework Bin folder to your system's PATH environment variable so that the tools can be located without typing the path to them on the command line. The general format of the `wsdl` command line is as follows:

`wsdl [options] (URL | Path)`

where *URL* specifies the HTTP address of a target Web server that you wish to search for any of the supported file types previously listed (excluding DISCOMAP files), and *Path* specifies the local file path to any of the supported file types listed.

6.10 CONSUMING WEB SERVICES

Many different types of applications can consume Web services, including traditional desktop applications, Web applications, or even other Web services. To consume a Web service, we must be able to do the following.

- Find the Web service
- Obtain its WSDL service contract
- Generate a proxy class with which to call the Web service
- Create an instance of the proxy class
- Call the methods exposed by the proxy

Recall that communication with a Web service is accomplished via messages that are delivered by a transport protocol. With no assistance from the platform, just to call a Web service, we would have to understand how to create and format SOAP messages as well as how to handle the delivery and receipt of these messages via HTTP.

Ideally, we would like the platform to provide this support for us. Of even greater benefit would be the ability to create an instance of a class that represented the Web service and called the methods, which in turn would carry out the necessary SOAP message generation and transportation activities required to communicate with the actual Web service.

Fortunately, this is exactly what happens when you create a Web service proxy class within the .NET Framework. The proxy class mimics the interfaces of the actual Web service and takes care of formatting appropriate SOAP messages to deliver requests to the Web service as well as process the responses that come back, such that calling a Web service is boiled down to the simple process of calling a method on a .NET class. This makes using Web services easy. An interesting attribute of ASP.NET Web services is that they can be referenced and called both as a native .NET class and as a Web service.

By generating and using a proxy class, however, we can reach our Web service via HTTP and SOAP. The benefit of the proxy class is that we can create an instance of the class and call its methods just like the native .NET assembly. The HTTP and SOAP plumbing required to call Web service methods is completely hidden from us within the proxy class (as it should be).

Why would we want to do this? Simple. We can distribute our Web service to any remote server, and all that we need is XML, HTTP, and SOAP to communicate with it. This greatly broadens the reach of our component to many different platforms, not just Windows platforms.

Let's take a look at the tools and techniques provided by ASP.NET and the .NET Framework to create an ASP.NET Web Forms application to consume our Web service.

Creating the Proxy

A proxy is a stand-in for the web service codes. Before using the web service, a proxy must be created. The proxy is registered with the client application. Then the client application makes the calls to the web service as it were using a local method.

The proxy takes the calls, wraps it in proper format and sends it as a SOAP request to the server. SOAP stands for Simple Object Access Protocol. This protocol is used for exchanging web service data.

When the server returns the SOAP package to the client, the proxy decodes everything and presents it to the client application.

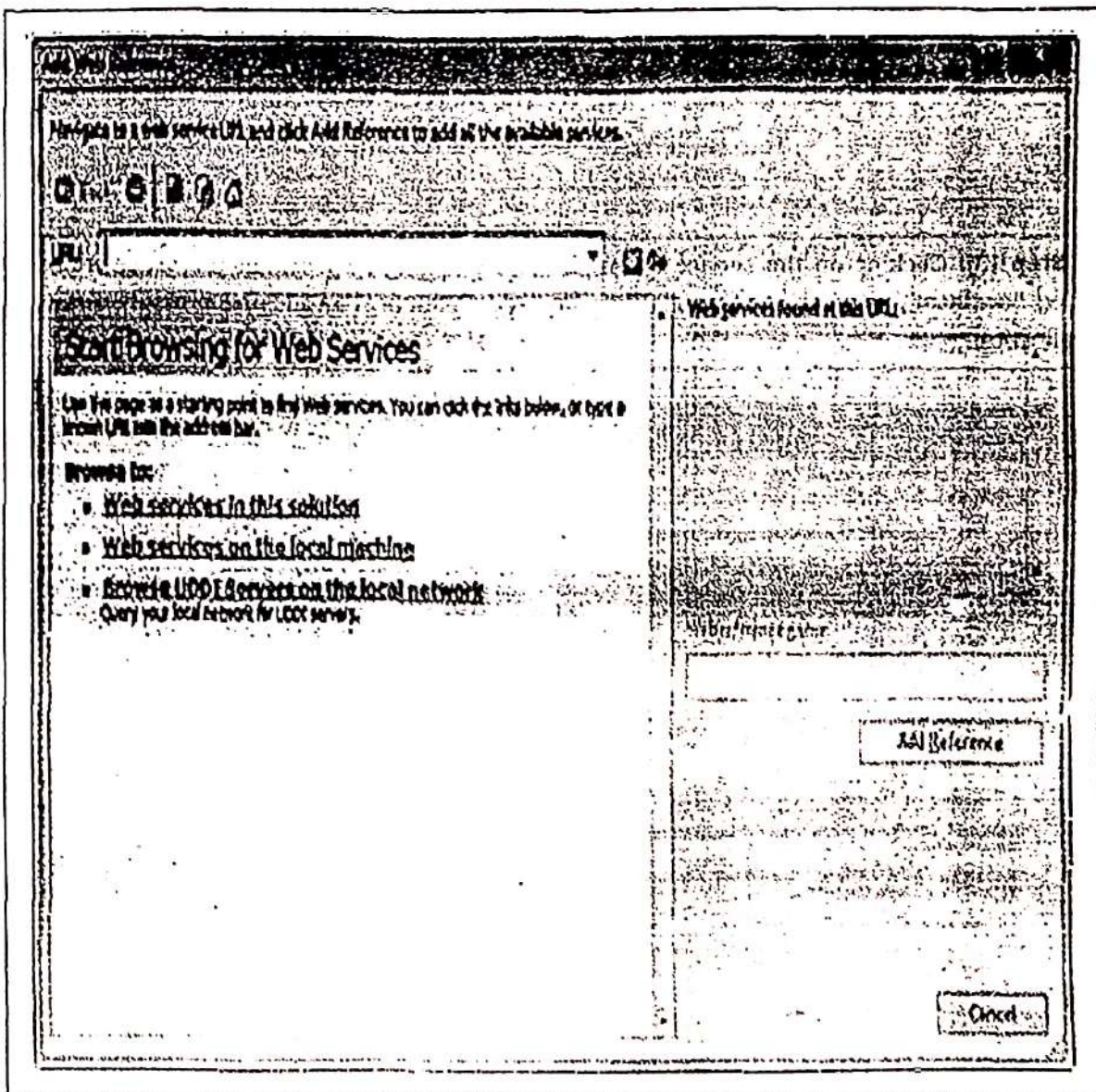
Before calling the web service using the `bin/service_Click`, a web reference should be added to

the application. This creates a proxy class transparently, which is used by the btnservice_Click event.

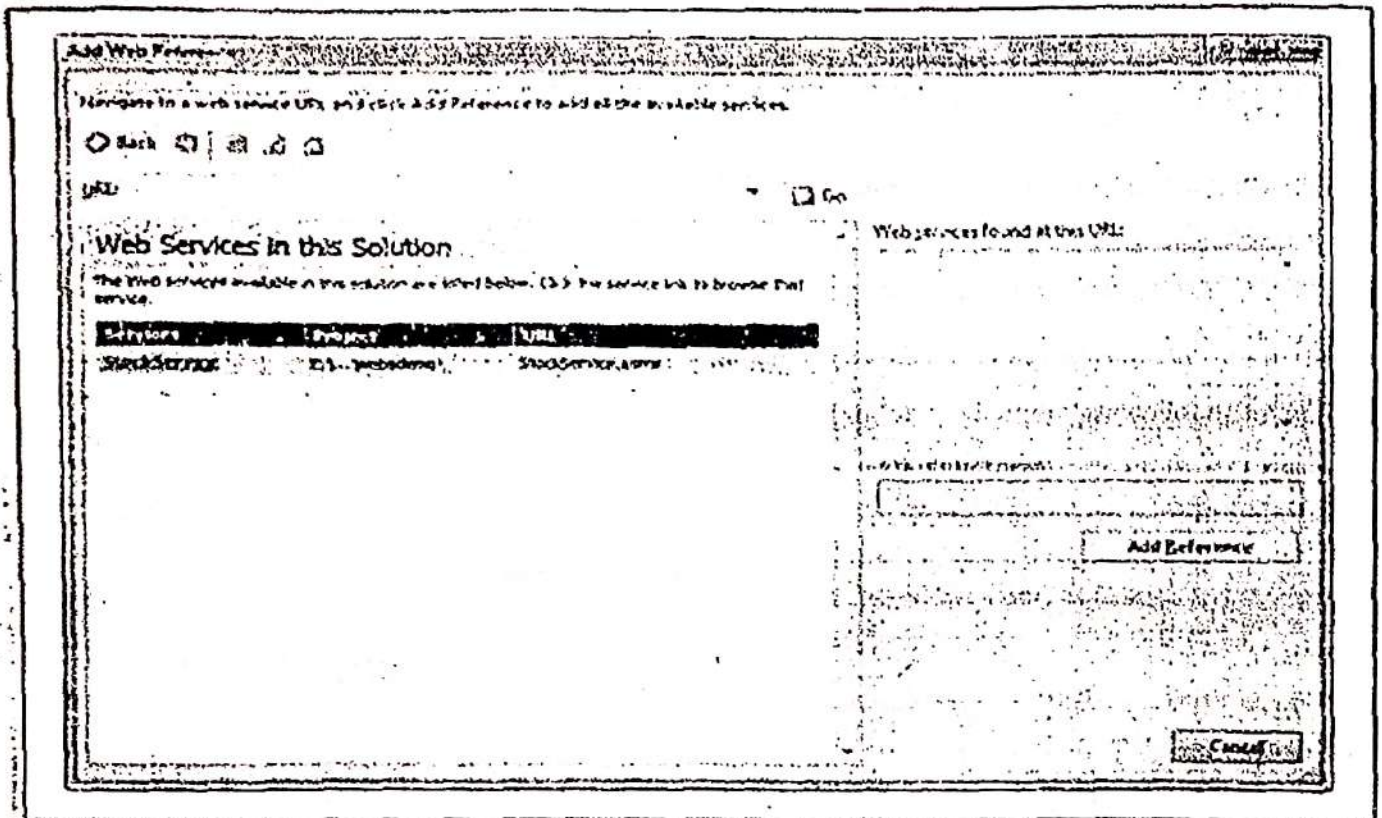
```
protected void btnservice_Click(object sender, EventArgs e)
{
    StockService proxy = new StockService();
    lblmessage.Text = String.Format("Current SATYAM Price: {0}",
    proxy.GetPrice("SATYAM").ToString());
}
```

Take the following steps for creating the proxy:

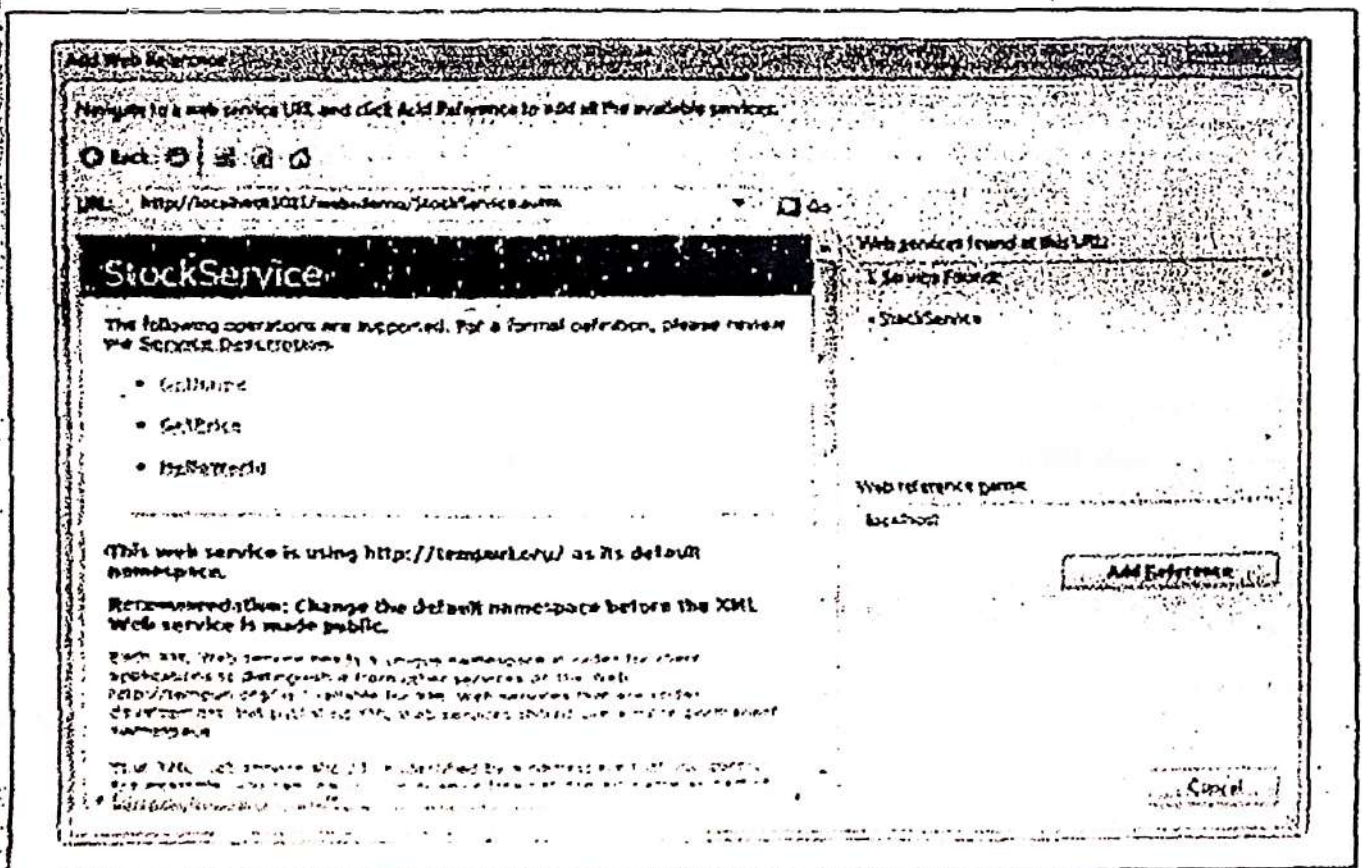
Step (1): Right click on the web application entry in the Solution Explorer and click on 'Add Web Reference'.



Step (2): Select 'Web Services in this solution'. It returns the StockService reference



Step (3): Clicking on the service opens the test web page. By default the proxy created is called 'localhost', you can rename it. Click on 'Add Reference' to add the proxy to the client application.



Include the proxy in the code behind file by adding:

```
using localhost;
```

Using Web Service

For using the web service, create a web site under the same solution that we have created in topic 6.7. This could be done by right clicking on the Solution name in the Solution Explorer. The web page calling the web service should have a label control to display the returned results and two button controls one for post back and another for calling the service.

The content file for the web application is as follows:

```
<%@ Page Language="C#"
    AutoEventWireup="true"
    CodeBehind="Default.aspx.cs"
    Inherits="wsclient._Default" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
<title>Untitled Page</title>
</head>
<body>
<form id="form1" runat="server">
    <div>
        <h3>Using the Stock Service</h3>
        <br />
        <br />
        <asp:Label ID="lblmessage" runat="server"></asp:Label>
        <br />
        <br />
        <asp:Button ID="btnpostback" runat="server"
            onclick="Button1_Click"
            Text="Post Back" Width="132px" />

        <asp:Button ID="btnservice" runat="server"
            onclick="btnservice_Click"
            Text="Get Stock" Width="99px" />
    </div>
```



```

</form>
</body>
</html>

```

The code behind file for the web application is as follows:

```

using System;
using System.Collections;
using System.Configuration;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Xml.Linq;

//this is the proxy
using localhost;

namespace wsclient
{
    public partial class _Default : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                lblmessage.Text = "First Loading Time: " +
                    DateTime.Now.ToLongTimeString();
            }
            else
            {
                lblmessage.Text = "PostBack at: " +
                    DateTime.Now.ToLongTimeString();
            }
        }
        protected void btnservice_Click(object sender, EventArgs e)
        {
            StockService proxy = new StockService();

```



```
lblmessage.Text = String.Format("Current SATYAM Price:{0}",  
    proxy.GetPrice("SATYAM").ToString());  
  
}  
  
}
```

REVIEW QUESTIONS

1. What area of development are Web Services best for? Is it possible to try out Web Services using only my local computer?
2. What are the various advantages & limitations of web services?
3. What are the various elements/Key Web Service Technologies in web service?
4. Explain primary infrastructure pieces which are specifically needed to support Web services.
5. What is the use of SOAP? What are the different capabilities provided by SOAP?
6. How to build a web service? Explain it with the help of example.
7. What are the different aspects of web service development and with it also explain different web service deployment tools.
8. Explain the technologies to enable potential consumers to locate the essential information they need in order to use Web service.
9. What are the different techniques for locating a Web service in ASP.NET?
10. What are the different steps to consume a web service? Explain it with the help of example using C# programming language?

